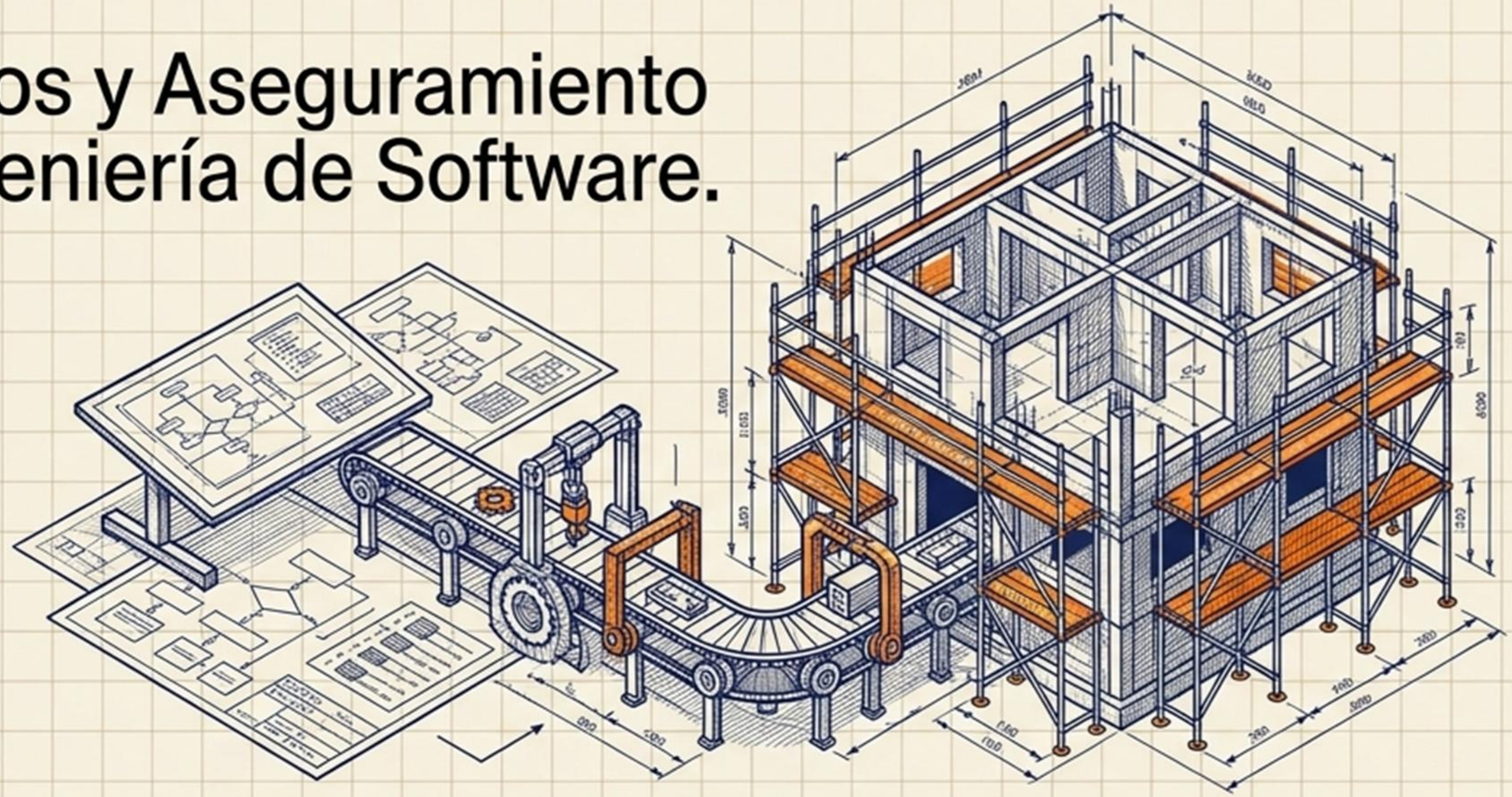
El Contrato de Calidad

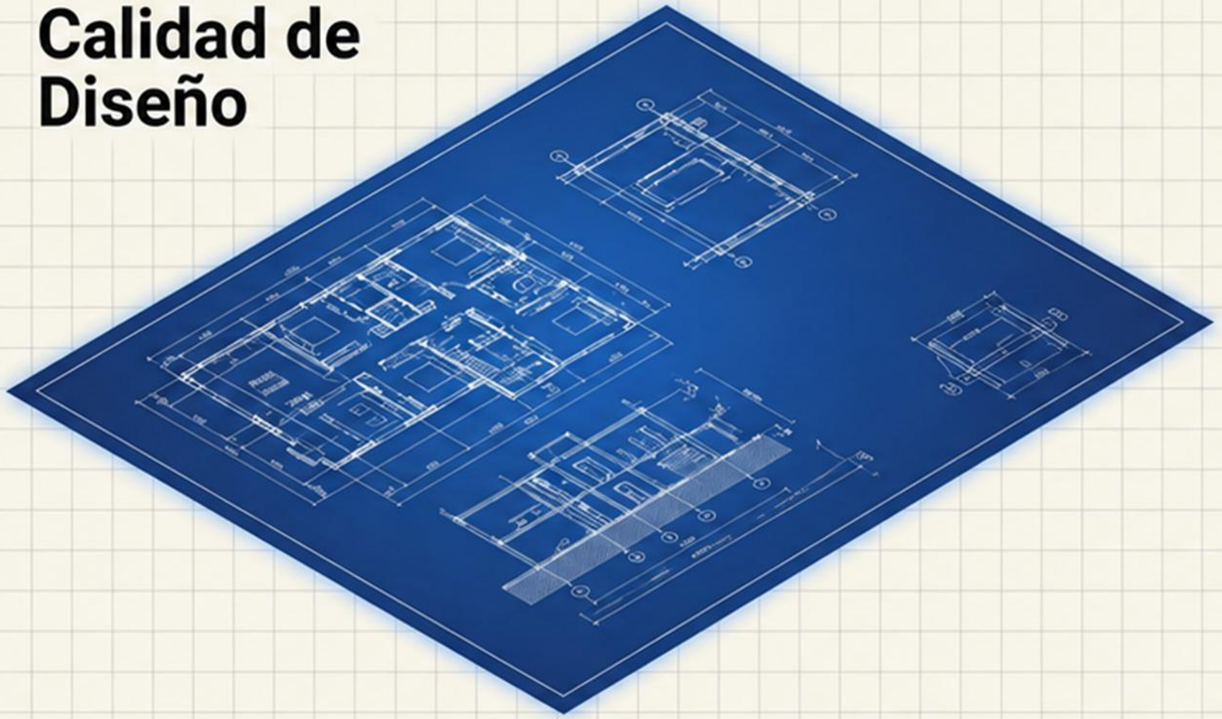
Conceptos y Aseguramiento en la Ingeniería de Software.



[KEY INSIGHT] La calidad no es un accidente; es una estructura que se diseña, se audita y se construye.

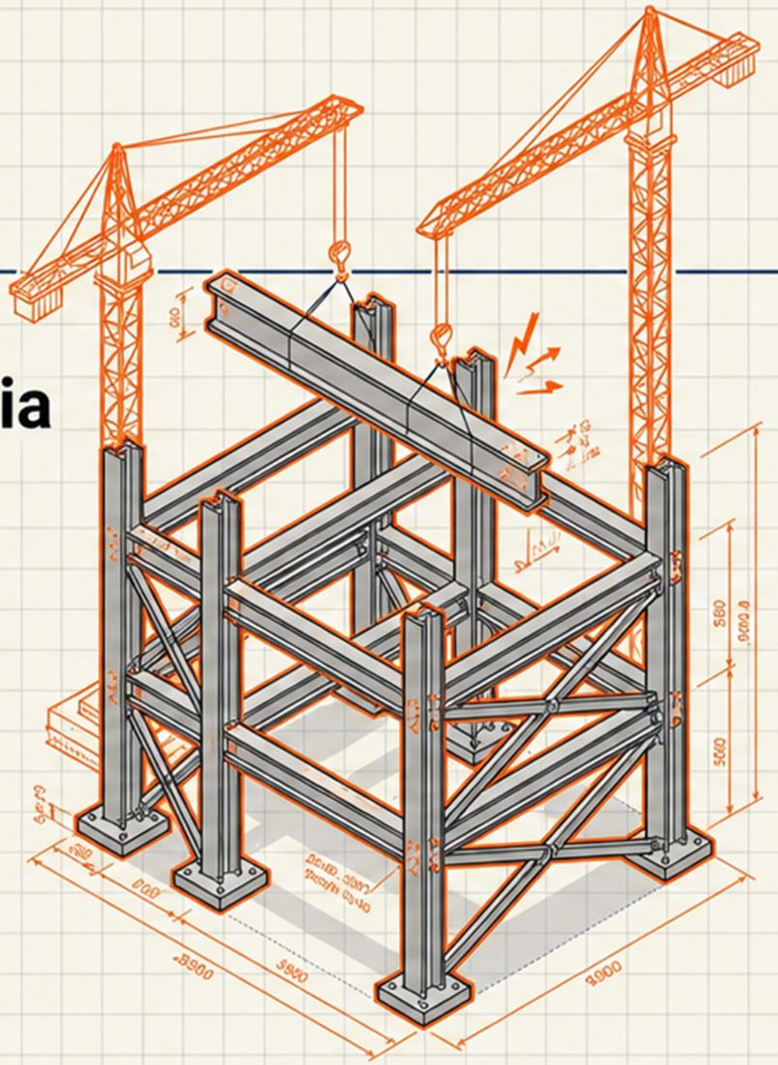
La Pregunta Incómoda: ¿Qué es la Calidad?

Calidad de Diseño



El grado en que el diseño cumple las funciones especificadas (¿El plano es correcto?).

Calidad de Concordancia



El grado en que la implementación se apega al diseño (¿Construimos según el plano?).

[KEY INSIGHT] Entregar a tiempo un sistema que nadie quiere es tan catastrófico como entregar tarde un sistema perfecto.

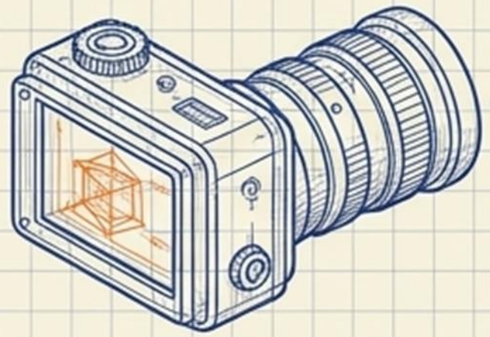
UIN RAFAELA

Acto I: Las Cinco Miradas de Garvin

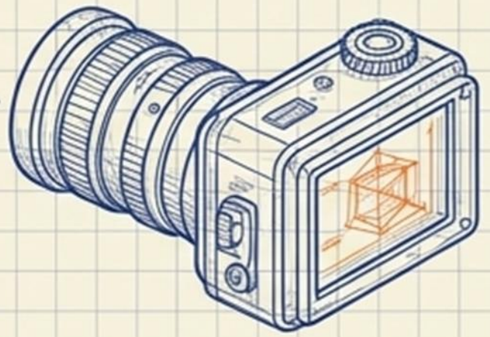
Trascendental:
Se reconoce cuando se ve (belleza/estética).

De Valor:
Cuánta utilidad ofrece en relación a su costo.

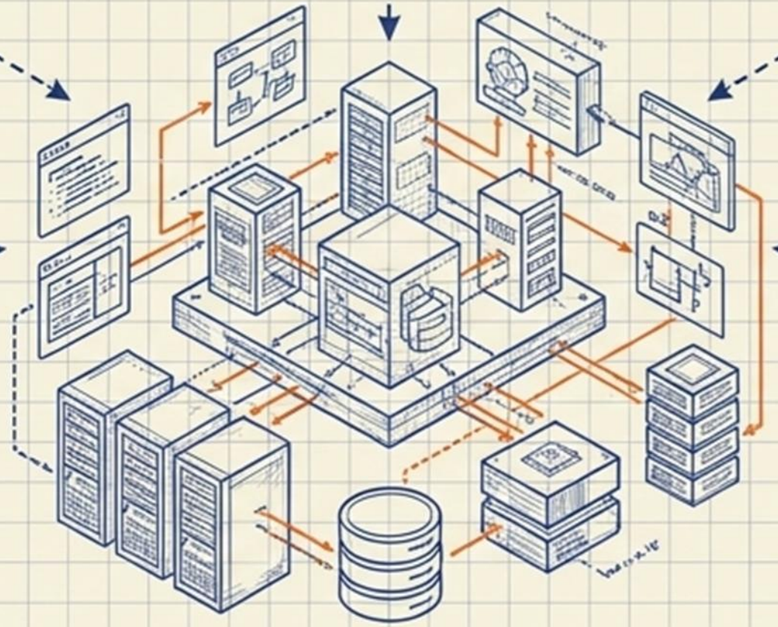
Del Usuario:
Satisface las necesidades prácticas del cliente.



Del Producto: Basado en características medibles e internas.



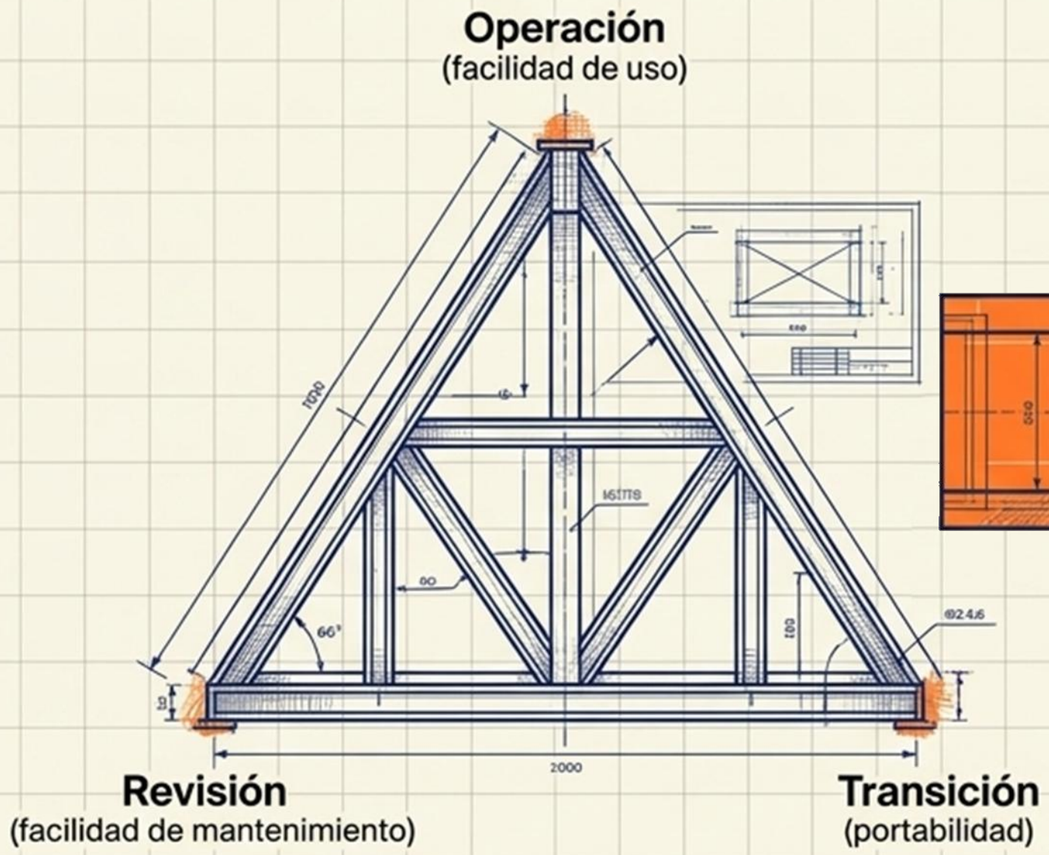
Del Fabricante: Cumple con las especificaciones internas (concordancia).



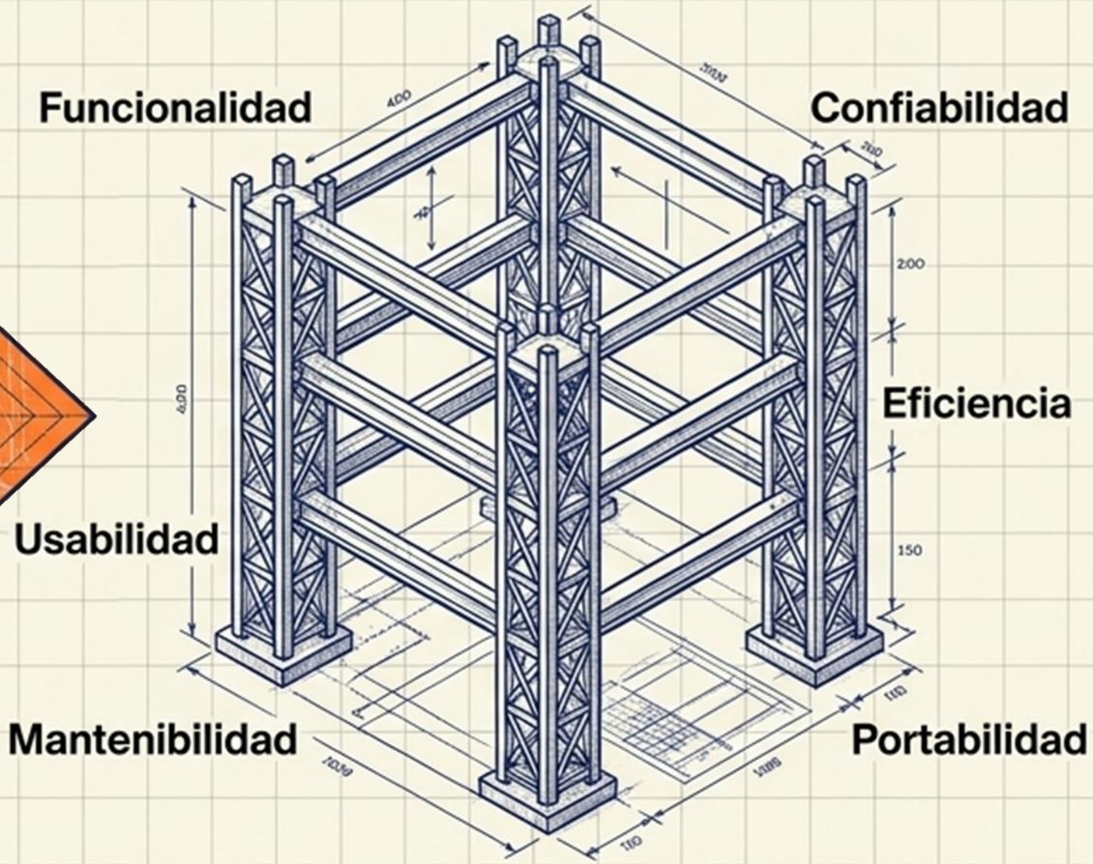
[KEY INSIGHT] La calidad es un poliedro. El desarrollador ve el código (Fabricante), pero el cliente ve el resultado (Usuario).

UTN FACULTAD REGIONAL RAFAELA

La Estructura de la Calidad: De McCall a ISO 9126



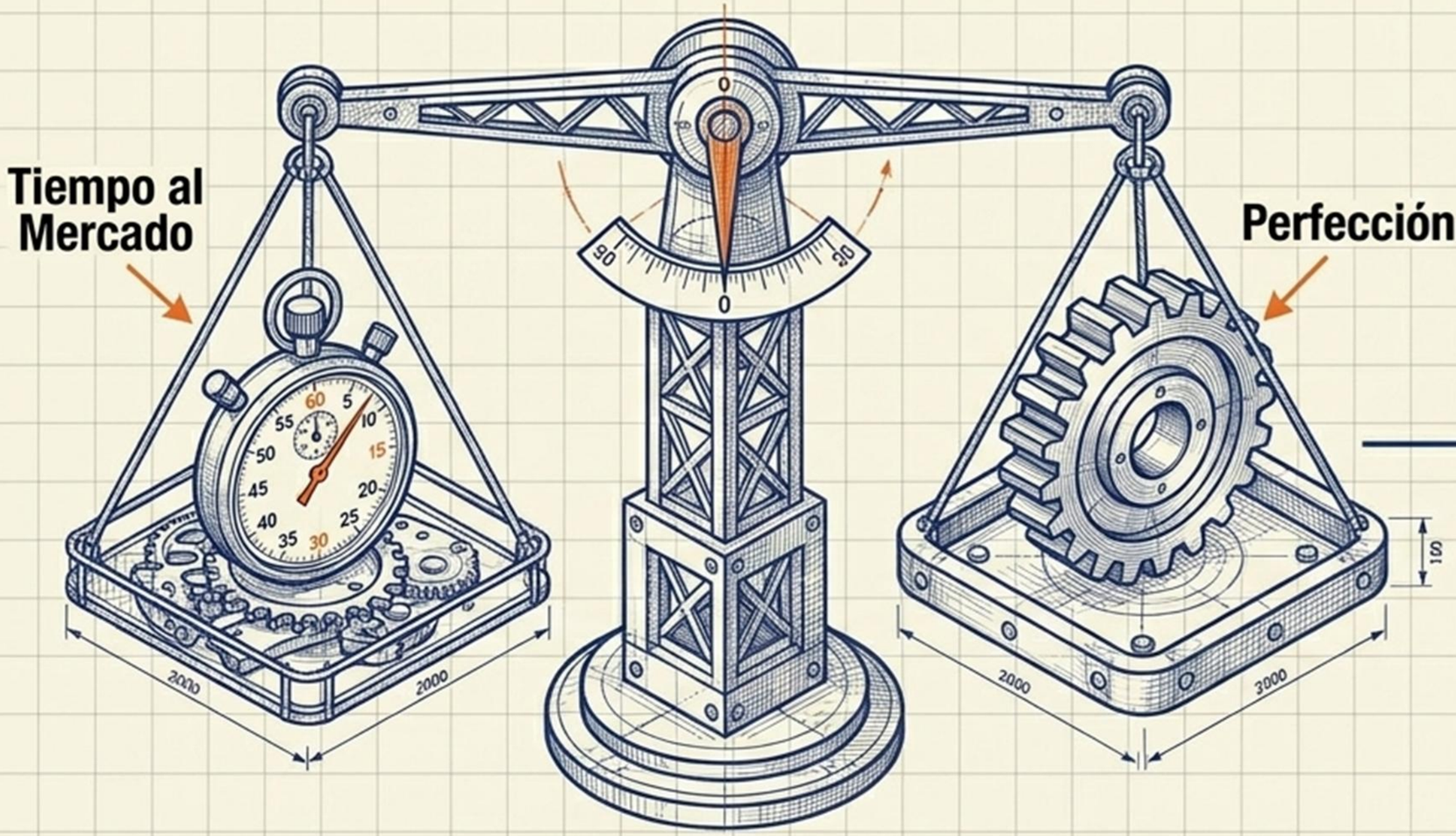
El Cimiento (McCall)



El Estándar Moderno (ISO 9126)

[KEY INSIGHT] Los estándares no inventan la calidad, proporcionan un lenguaje común para medirla sin ambigüedades.

Acto II: El Dilema del Software Suficientemente Bueno



La Paradoja de Meyer: Buscar la perfección agota el presupuesto y pierde la ventana de mercado.

La Trampa: Entregar algo “suficientemente bueno” a menudo significa trasladar el trabajo de depuración al cliente.

La Ley de Meskimen: Nunca hay tiempo para hacerlo bien, pero siempre hay tiempo para hacerlo otra vez.

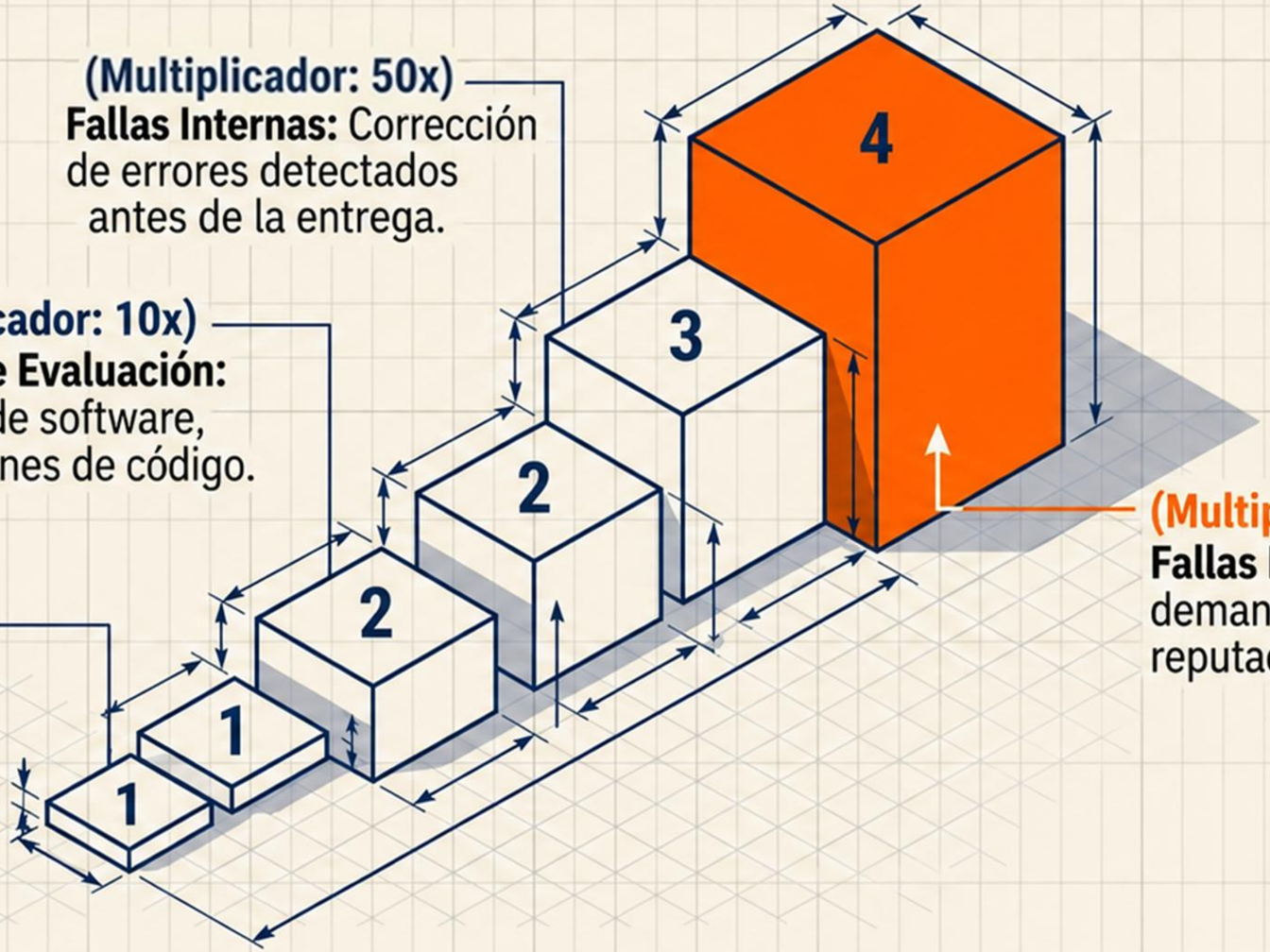
[KEY INSIGHT] La calidad suficientemente buena para una startup puede ser un suicidio operativo para un sistema crítico.

El Costo Exponencial de la Calidad

(Costo Base: 1x)
Costos de Prevención: Planificación, revisiones técnicas, capacitación.

(Multiplicador: 10x)
Costos de Evaluación: Pruebas de software, inspecciones de código.

(Multiplicador: 50x)
Fallas Internas: Corrección de errores detectados antes de la entrega.



(Multiplicador: 100x+)
Fallas Externas: Soporte, demandas, pérdida de reputación tras el lanzamiento.

[KEY INSIGHT] El código defectuoso es una deuda con intereses usureros; cuanto más tarde se paga, más cara resulta.

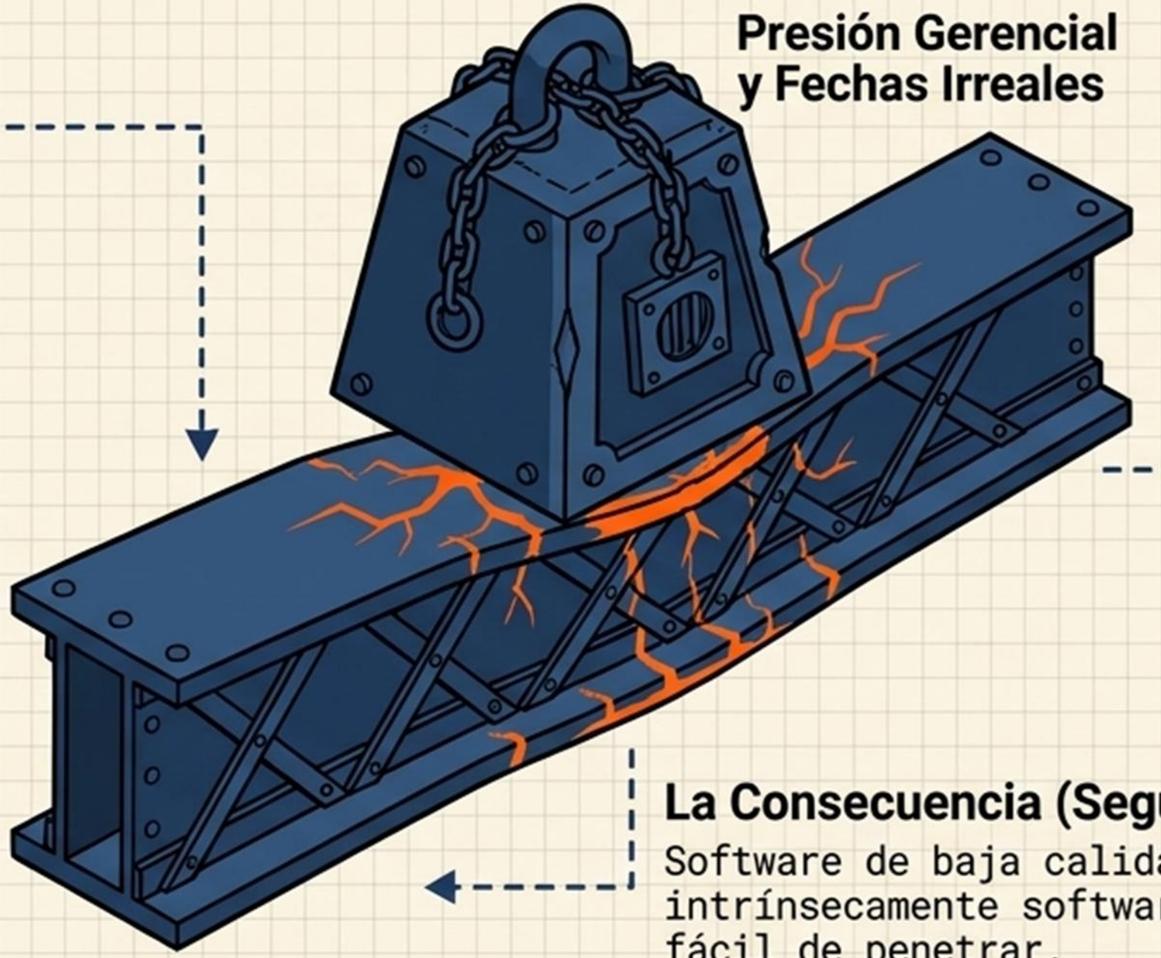
UTN FACULTAD REGIONAL RAFAELA

El Factor Humano: Riesgo y Seguridad

El Origen (Decisiones de Administración):

El optimismo ciego al estimar fechas obliga a saltar fases de revisión.

Presión Gerencial y Fechas Irreales



El Resultado (Vulnerabilidades):

Fallas de arquitectura y defectos

Fallas de arquitectura (defectos estructurales)

Errores de implementación (bugs)

Arquitectura del Software

La Consecuencia (Seguridad):

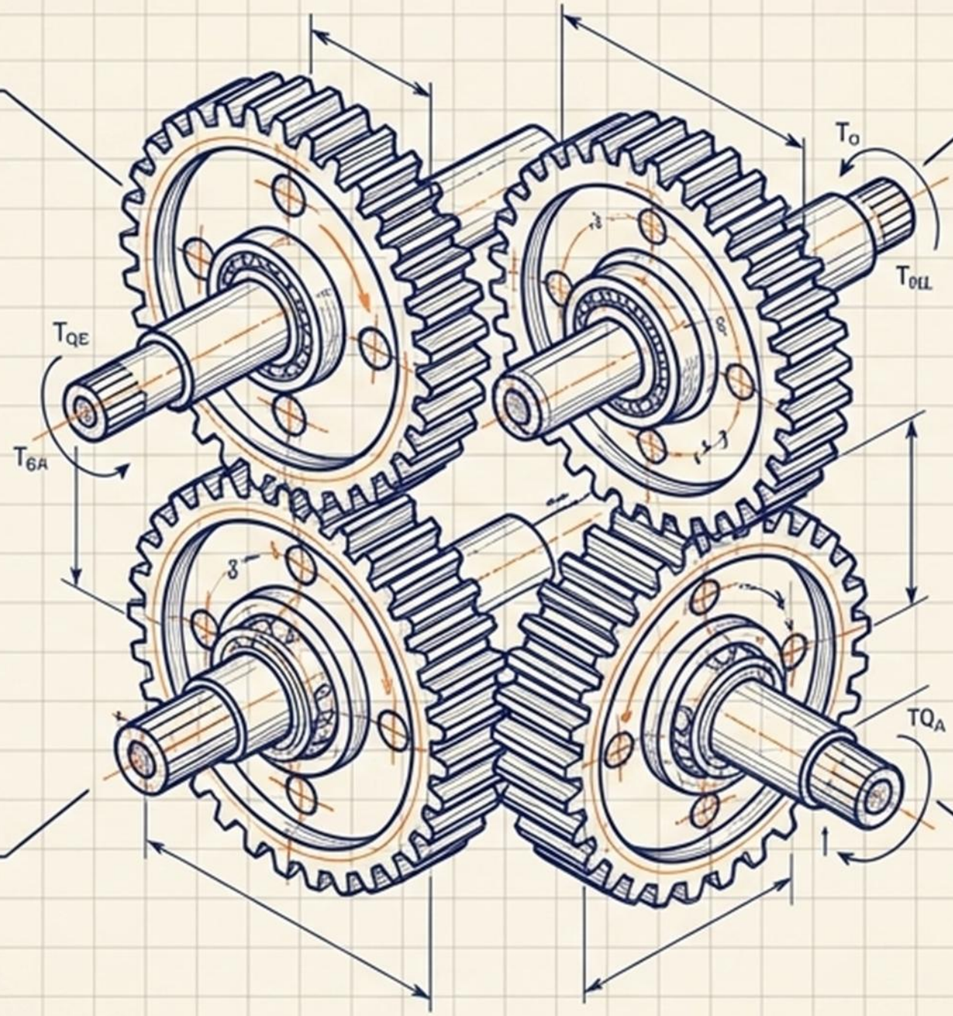
Software de baja calidad es intrínsecamente software fácil de penetrar.

[KEY INSIGHT] Para construir un sistema seguro, se debe comenzar con un diseño de alta calidad. La seguridad es un subproducto de la calidad.

Acto III: Los Cuatro Engranajes para Lograrla

Métodos de Ingeniería:
Análisis sólido y diseño arquitectónico correcto.

Gestión de Proyectos (PM):
Estimación realista y control de riesgos.



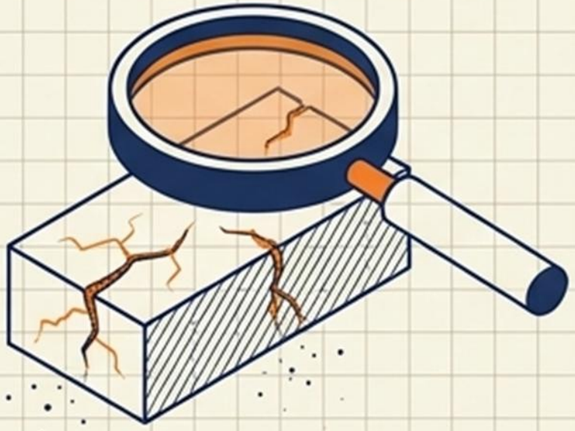
Control de Calidad (QC):
Acciones reactivas para detectar errores.

Aseguramiento (QA):
Supervisión proactiva del proceso completo.

[KEY INSIGHT] Si un solo engranaje falla (ej. una mala gestión del proyecto), la máquina entera produce software defectuoso

La Distinción Crucial: Control vs. Aseguramiento

Control de Calidad (QC)



Rol: El Inspector Final de la Obra.
Enfoque: Reactivo. Enfocado en el producto.
Acción: Ejecuta pruebas, detecta bugs, inspecciona código.

Aseguramiento de Calidad (QA)

Rol: El Auditor de Procedimientos.
Enfoque: Preventivo. Enfocado en el proceso.
Acción: Define estándares, reporta a gerencia, asegura que el QC se haga bien.

[KEY INSIGHT] El QC encuentra los errores; el QA asegura que exista un sistema que impida que esos errores se repitan.

Acto IV: La Maquinaria del ACS (SQA)

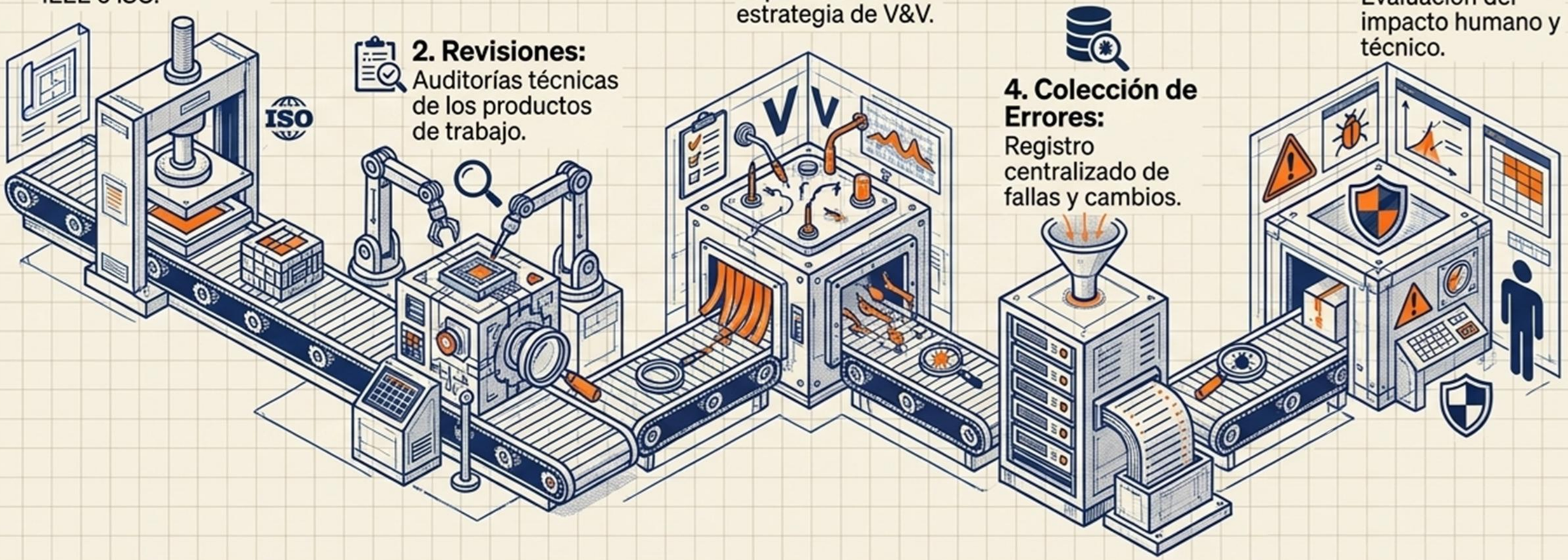
1. Estándares:
Adopción de normas IEEE o ISO.

2. Revisiones:
Auditorías técnicas de los productos de trabajo.

3. Pruebas:
Supervisión de la estrategia de V&V.

4. Colección de Errores:
Registro centralizado de fallas y cambios.

5. Seguridad y Riesgos:
Evaluación del impacto humano y técnico.



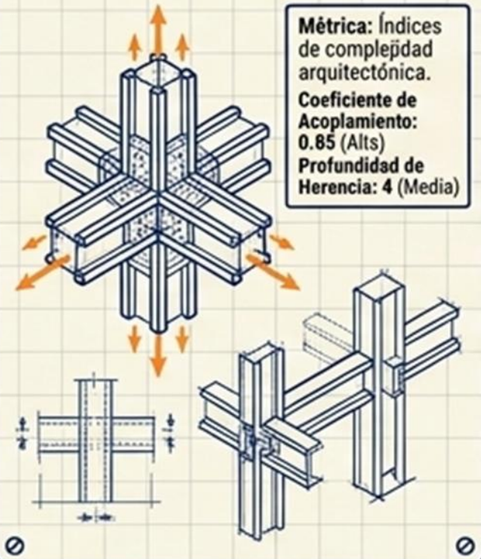
[KEY INSIGHT] El grupo de ACS actúa como el representante del cliente infiltrado dentro del equipo de desarrollo.

El Tablero de Medición: De Metas a Métricas

Metas de Requerimientos: Exactitud y completitud

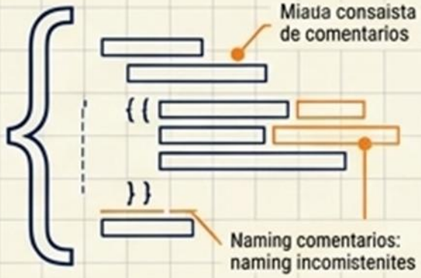
Métrica:
Ambigüedad detectada en revisiones.

Metas de Diseño: Robustez estructural.

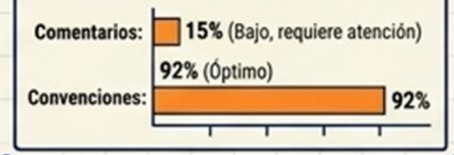


Métrica: Índices de complejidad arquitectónica.
Coeficiente de Acoplamiento: 0.85 (Alts)
Profundidad de Herencia: 4 (Media)

Metas de Código: Mantenibilidad.



Métrica:
Porcentaje de comentarios internos, convenciones de nomenclatura.



Eficacia del Control: Rendimiento del equipo.

Métrica:
Errores detectados vs. esfuerzo requerido para corregir.

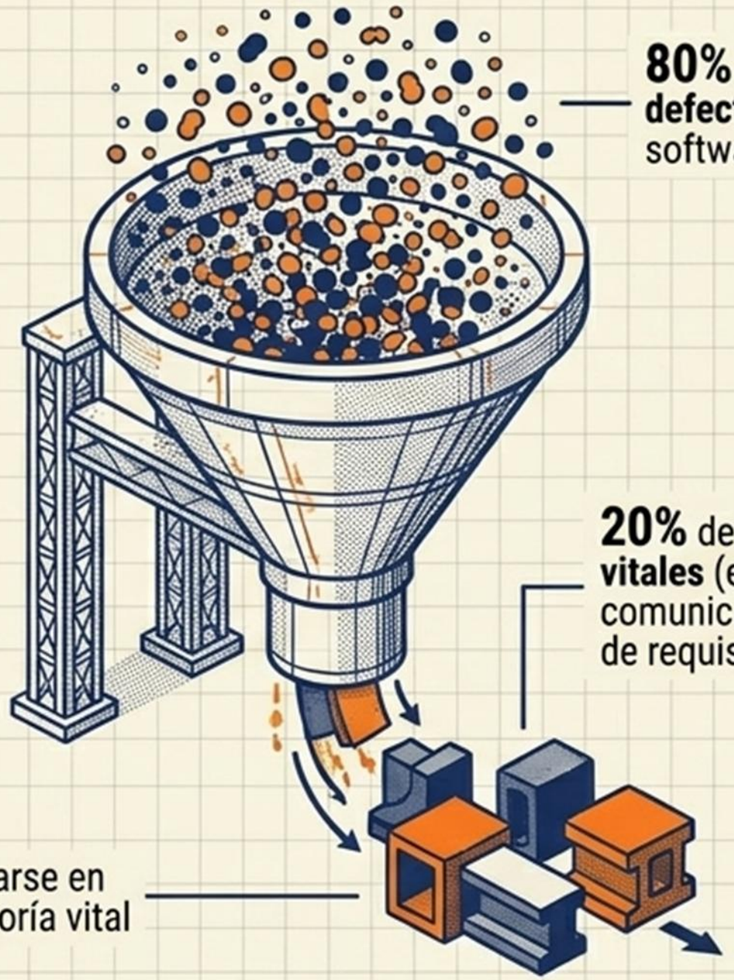
Relación E/E: 0.25

[KEY INSIGHT] Lo que no se puede medir, no se puede asegurar.
La calidad debe traducirse en atributos observables.

UTN FACULTAD REGIONAL RAFAELA

El Enfoque Cuantitativo: Pareto y Seis Sigma

El Principio de Pareto (80/20)

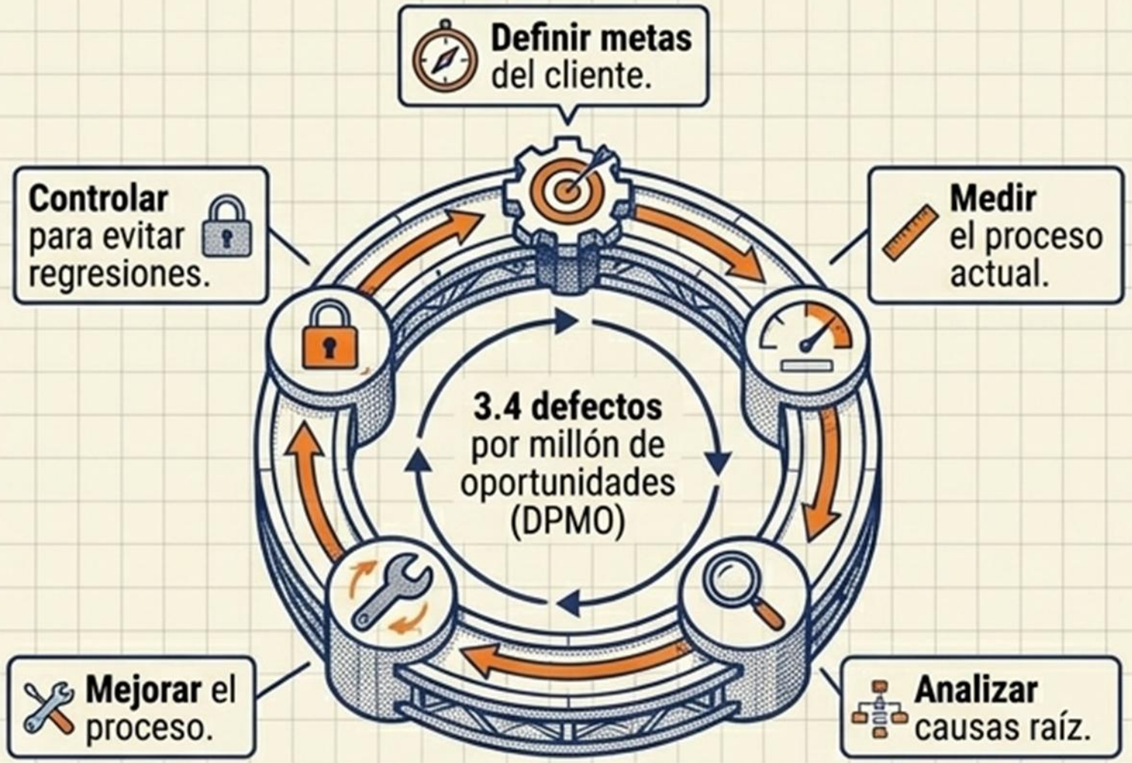


80% de los **defectos** del software

20% de las **causas vitales** (ej. mala comunicación de requisitos).

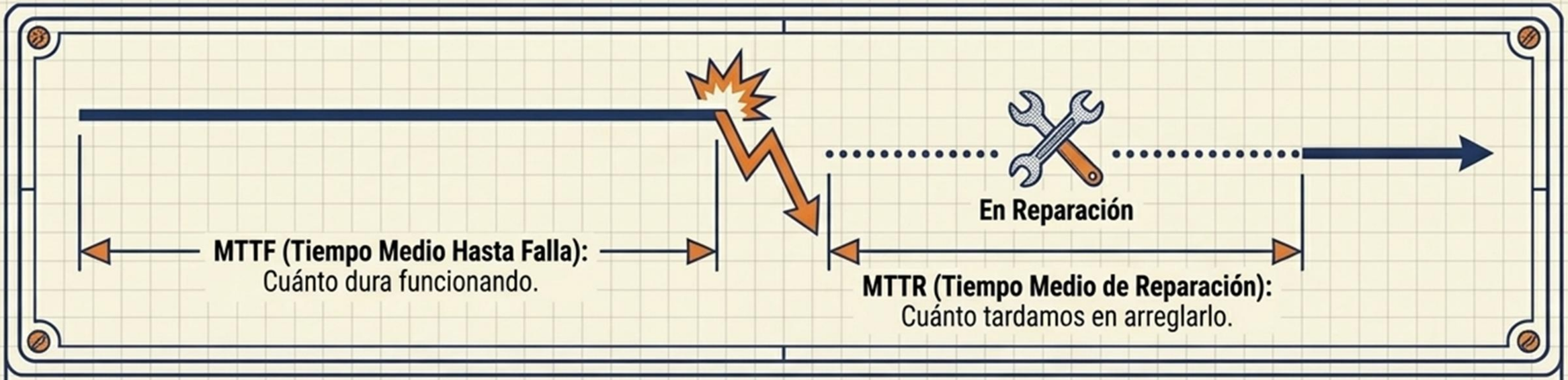
Enfocarse en la minoría vital

Metodología Seis Sigma (3.4 defectos por millón)



[KEY INSIGHT] Pasa tu tiempo arreglando las causas que realmente importan, no apagando incendios individuales.

La Ecuación de la Confiabilidad



Ecuación MTBF = MTTF + MTTR

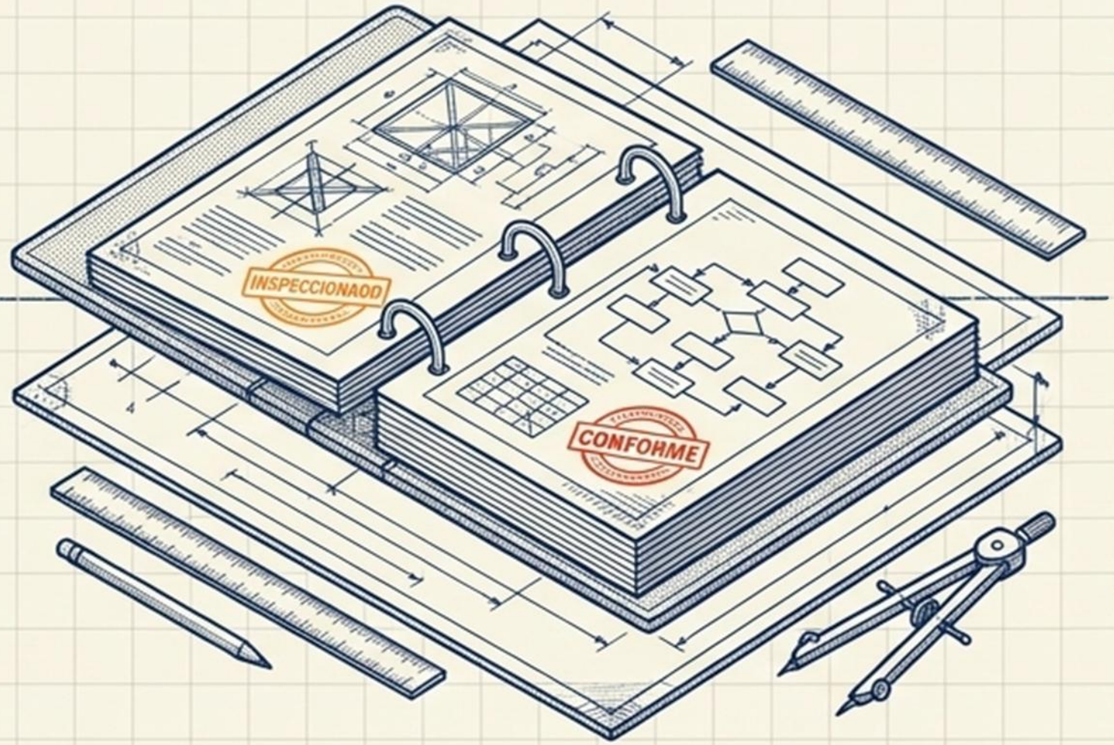
$$\text{Disponibilidad} = \frac{\text{MTTF}}{\text{MTBF}} \times 100\%$$

(Probabilidad de que el sistema opere cuando se le necesite).

[KEY INSIGHT] Un software altamente confiable (poco propenso a fallar) es inútil si su tiempo de reparación (MTTR) es prohibitivamente largo.

El Manual de Normas: Plan ACS e ISO 9000

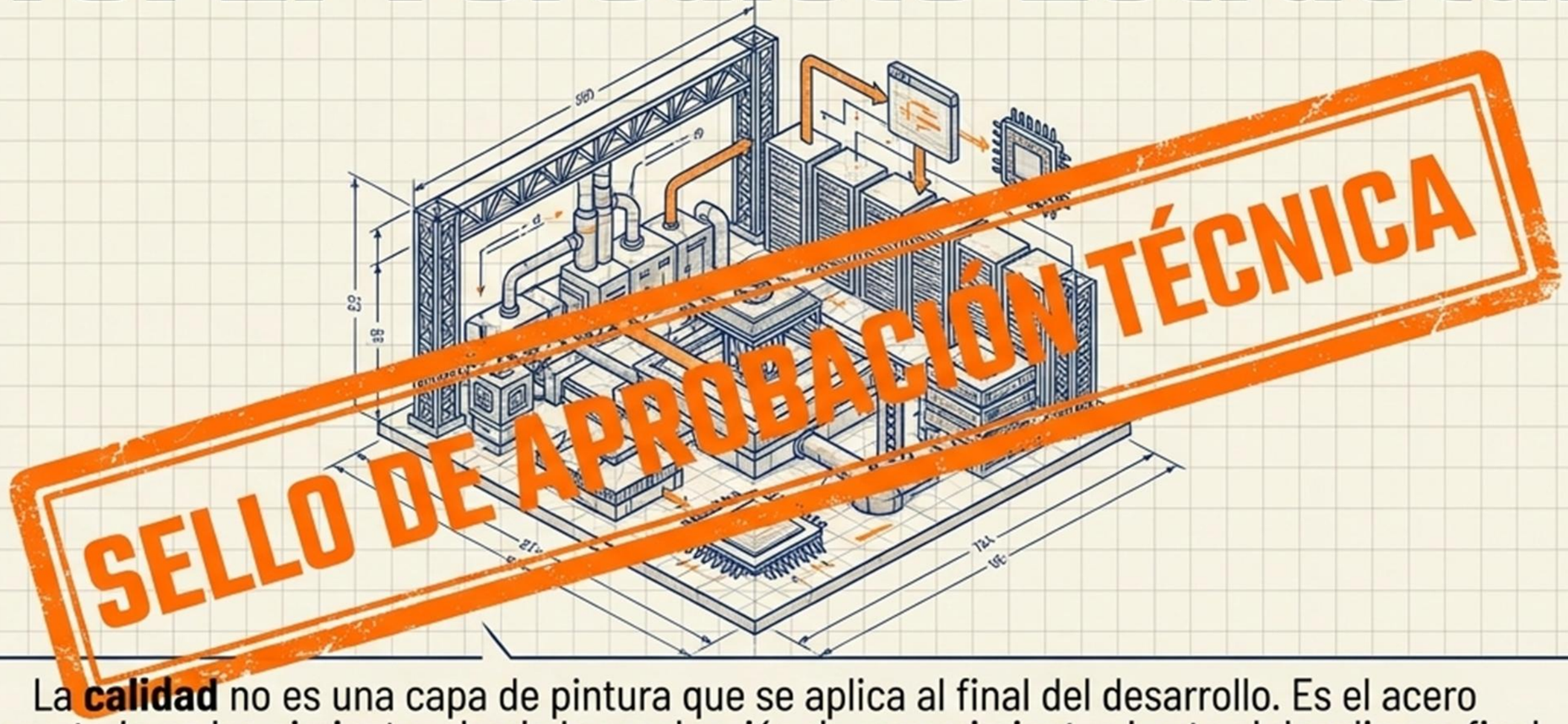
El Plan de ACS (SQA Plan): El mapa de ruta para cada proyecto específico.
Define productos a auditar, normas aplicables, herramientas, métodos y responsabilidades (basado en IEEE).



Norma ISO 9000/9001: Certifica que la organización tiene un sistema de administración de calidad formal a nivel empresarial. Abarca todo el ciclo de vida del producto.

[KEY INSIGHT] La certificación no asegura un software libre de fallas, pero garantiza que la empresa no depende de la improvisación.

Cierre: El Veredicto Estructural



La **calidad** no es una capa de pintura que se aplica al final del desarrollo. Es el acero incrustado en los cimientos desde la recolección de requerimientos hasta el despliegue final.

¿Cómo garantizamos que esta estructura resista el peso de la realidad?
A través de la ejecución rigurosa de las Pruebas de Software (Testing).

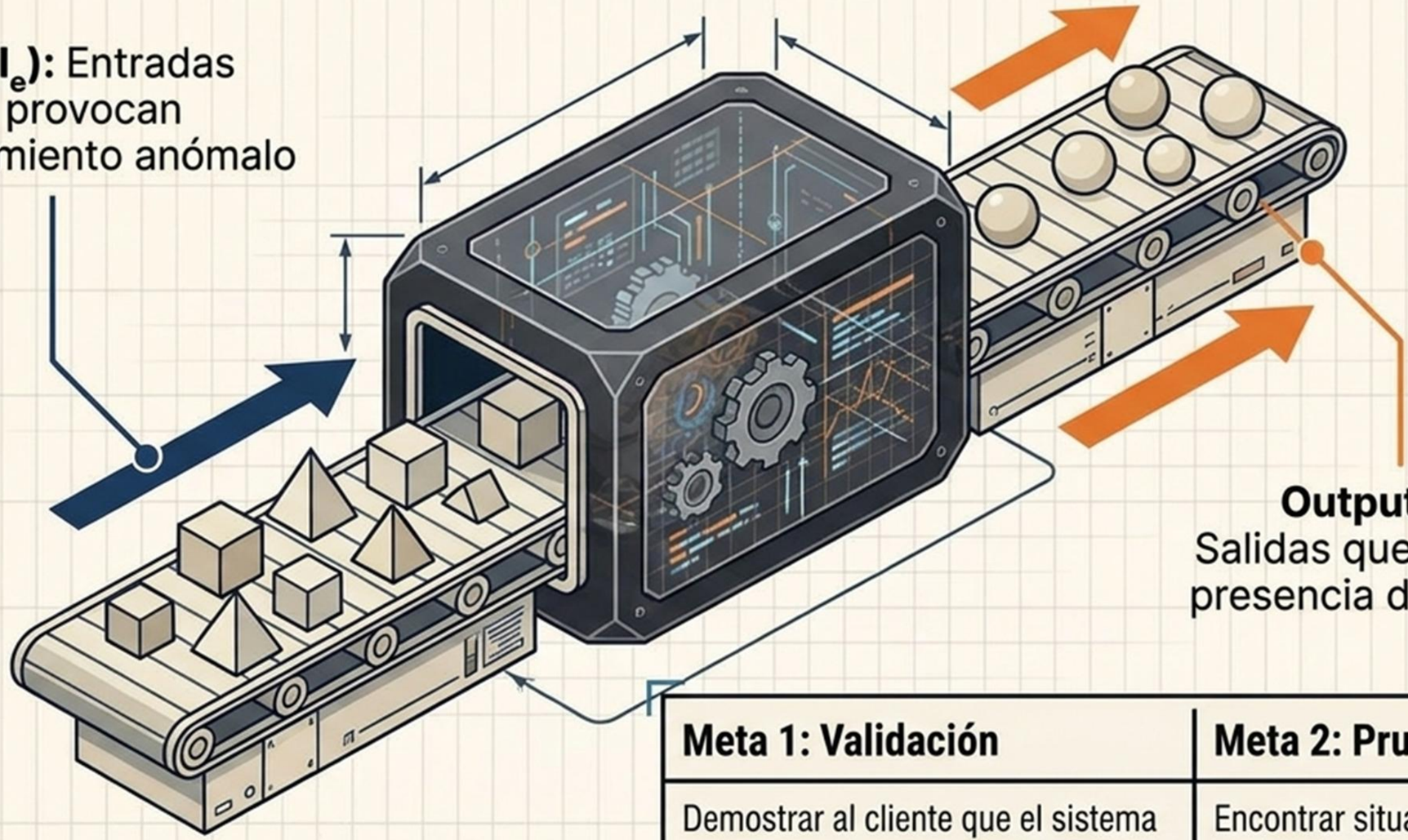
[KEY INSIGHT] La calidad no se agrega al final: se diseña desde el principio.

El Mapa de la Cacería: Pruebas de Software

Estrategias y niveles de inspección según Ian Sommerville.
Una guía visual de ingeniería de calidad.

El Modelo de Caja Negra: Las Dos Metas

Input (I_e): Entradas que provocan comportamiento anómalo



Output (O_e): Salidas que revelan la presencia de defectos

Meta 1: Validación	Meta 2: Prueba de Defectos
Demostrar al cliente que el sistema cumple con los requerimientos previstos (Comportamiento correcto).	Encontrar situaciones donde el comportamiento es incorrecto o indeseable (Cazar el error).

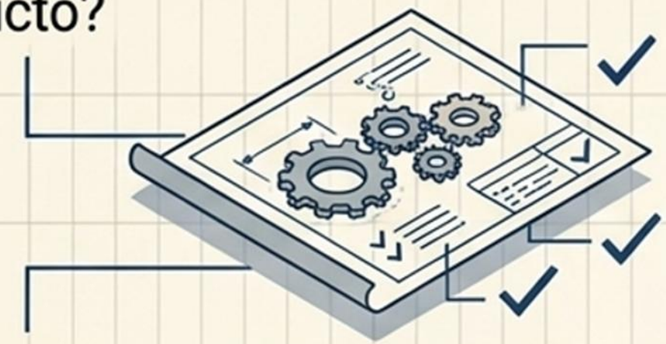
> “Las pruebas pueden mostrar sólo la presencia de errores, mas no su ausencia.”
> — Edsger Dijkstra

Por qué seguimos probando: Probar nunca garantiza un software perfecto, pero sigue siendo nuestra herramienta de verificación práctica más rigurosa para mitigar el riesgo operacional.

La Distinción Crucial

Verificación

Pregunta Guía: ¿Construimos bien el producto?



Enfoque: Comprobar que el software cumple con su funcionalidad técnica y requerimientos no funcionales especificados.

Validación

Pregunta Guía: ¿Construimos el producto correcto?

Enfoque: Garantizar que el software cumple verdaderamente con las expectativas y necesidades reales del cliente (las cuales a veces difieren del documento de diseño).

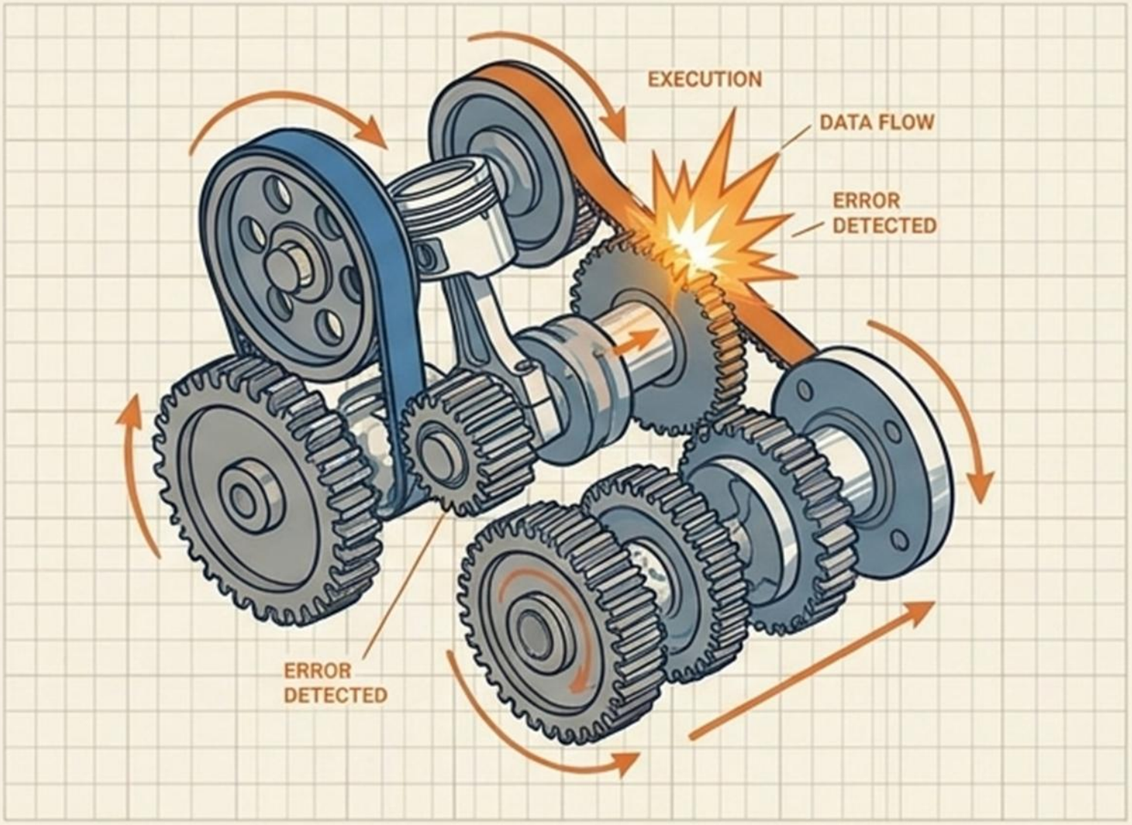
V&V (Verificación y Validación) es un proceso continuo a través de todo el ciclo de vida, no solo una fase final.

El Enfoque Dual: Inspecciones vs. Pruebas

Inspecciones (V&V Estática)

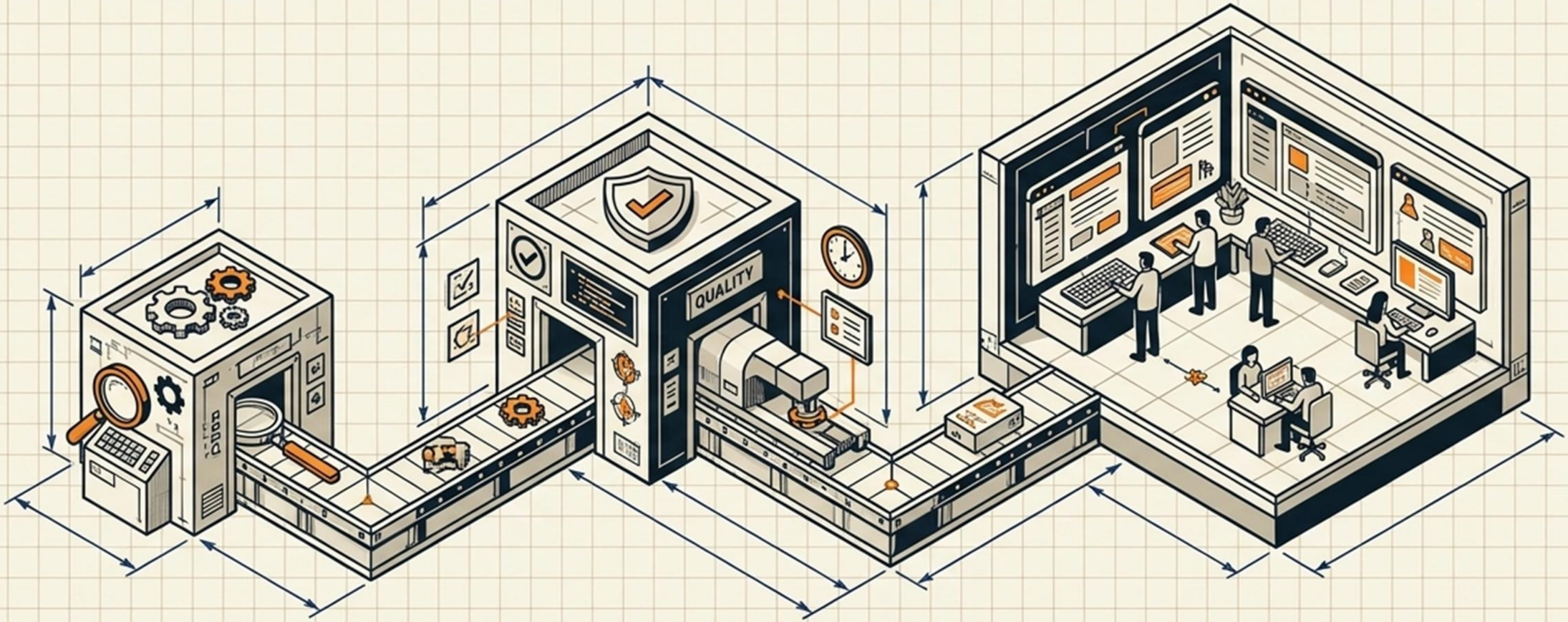
- **No requiere** ejecutar el sistema.
- **Se enfoca en:** Código fuente, requerimientos, modelos de diseño.
- **Ventaja:** Un error no enmascara a otro. Encuentra problemas de estilo o algoritmos ineficientes.

Pruebas Dinámicas (V&V Dinámica)



- **Requiere** ejecutar el programa con datos de prueba artificiales.
- **Se enfoca en:** Comportamiento del sistema, rendimiento, cuellos de botella.
- **Ventaja:** Detecta interacciones inesperadas y problemas de temporización en tiempo real.

Acto I: El Mapa de las Tres Estaciones



1. Pruebas de Desarrollo

Actor: El equipo de desarrollo.
Objetivo: Descubrir bugs y defectos técnicos durante la construcción.

2. Pruebas de Versión (Release)

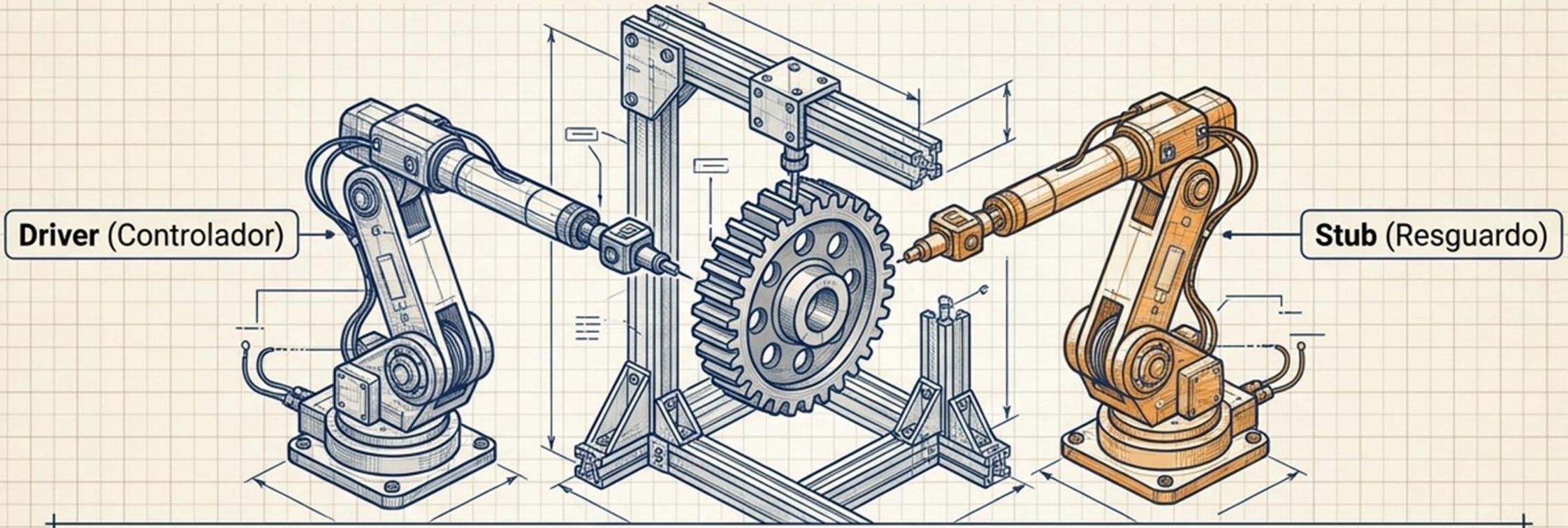
Actor: Equipo de prueba independiente.
Objetivo: Comprobar que el sistema completo cumple con los requerimientos de los participantes.

3. Pruebas de Usuario

Actor: Usuarios reales o potenciales.
Objetivo: Validar el sistema en su entorno operacional real.

Estación 1: Pruebas de Unidad

Probar piezas atómicas aisladas (funciones, métodos, clases).

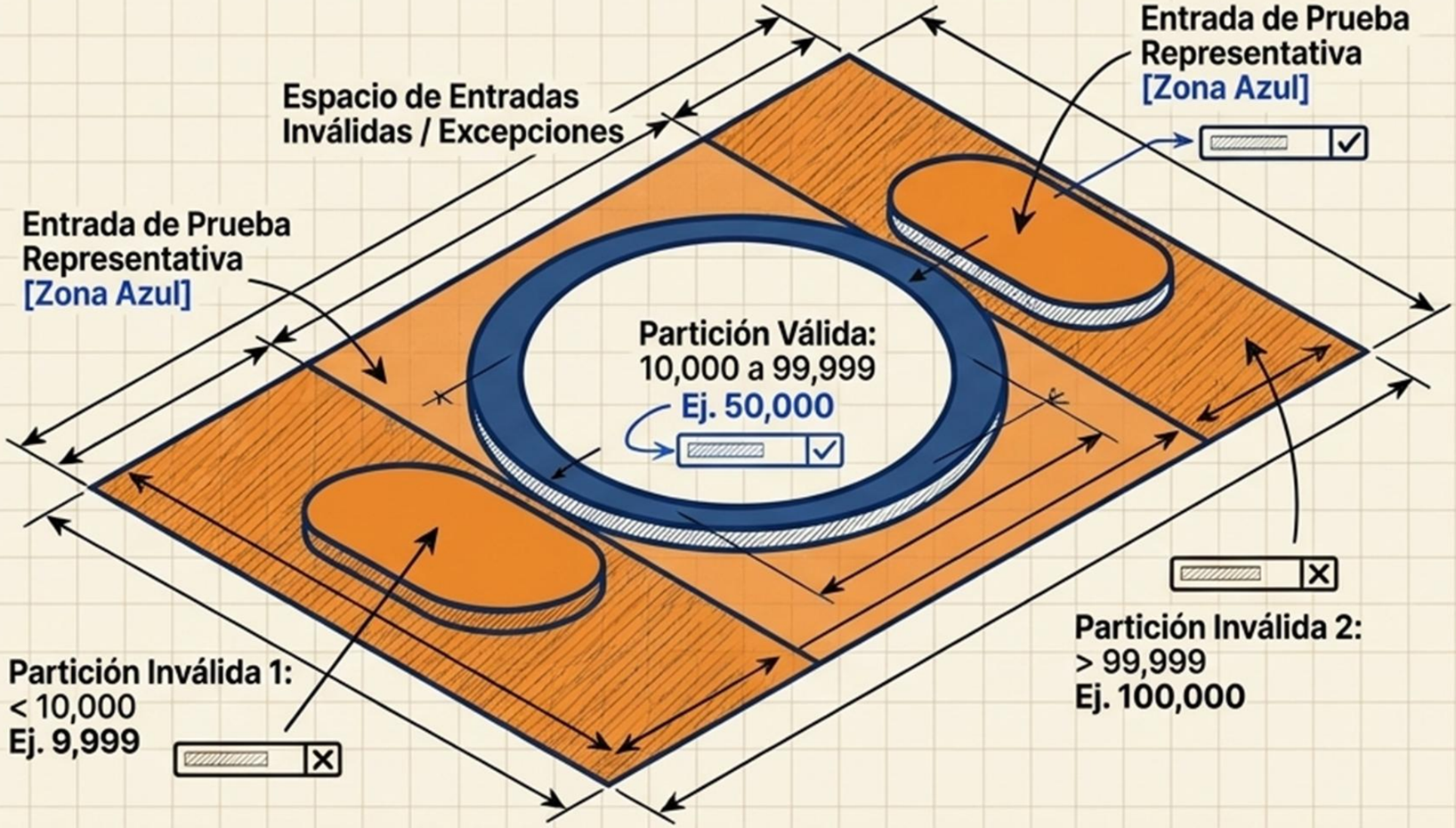


El Ciclo de Automatización (Frameworks tipo xUnit)

1. **Configuración (Setup)**: Inicializar el sistema con los datos de entrada previstos.	2. **Llamada (Execute)**: Ejecutar el objeto o método a probar.	3. **Declaración (Assert)**: Comparar el resultado real con el resultado esperado. Si es idéntico, pasa (Azul). Si difiere, falla (Naranja).

Táctica de Caza: Partición de Equivalencia

Agrupar entradas en clases donde el programa se comporta de forma idéntica. Se prueba solo un representante por clase para optimizar esfuerzo.

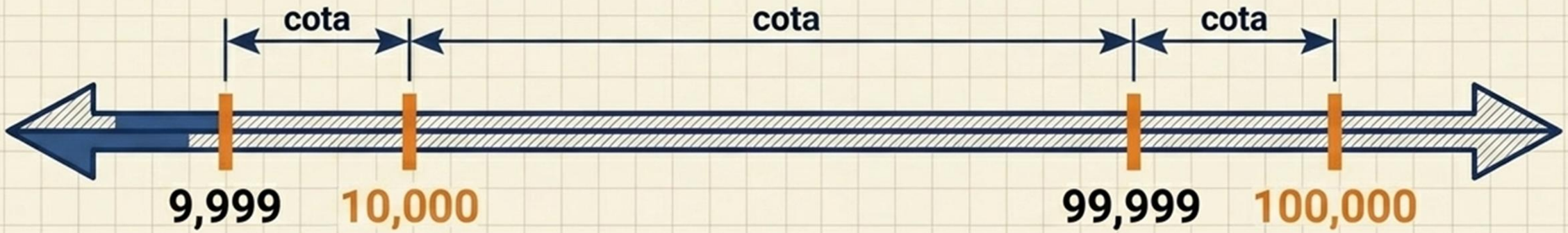


Ejemplo Práctico: Un campo que acepta números del 10,000 al 99,999.

- **Partición Inválida 1:** < 10,000 (Ej. 9,999) [Zona Naranja]
- **Partición Válida:** 10,000 a 99,999 (Ej. 50,000) [Zona Azul]
- **Partición Inválida 2:** > 99,999 (Ej. 100,000) [Zona Naranja]

Táctica de Caza: Valores de Frontera

Los errores de programación (bugs) se agrupan en los bordes de las particiones debido a confusiones con operadores lógicos.

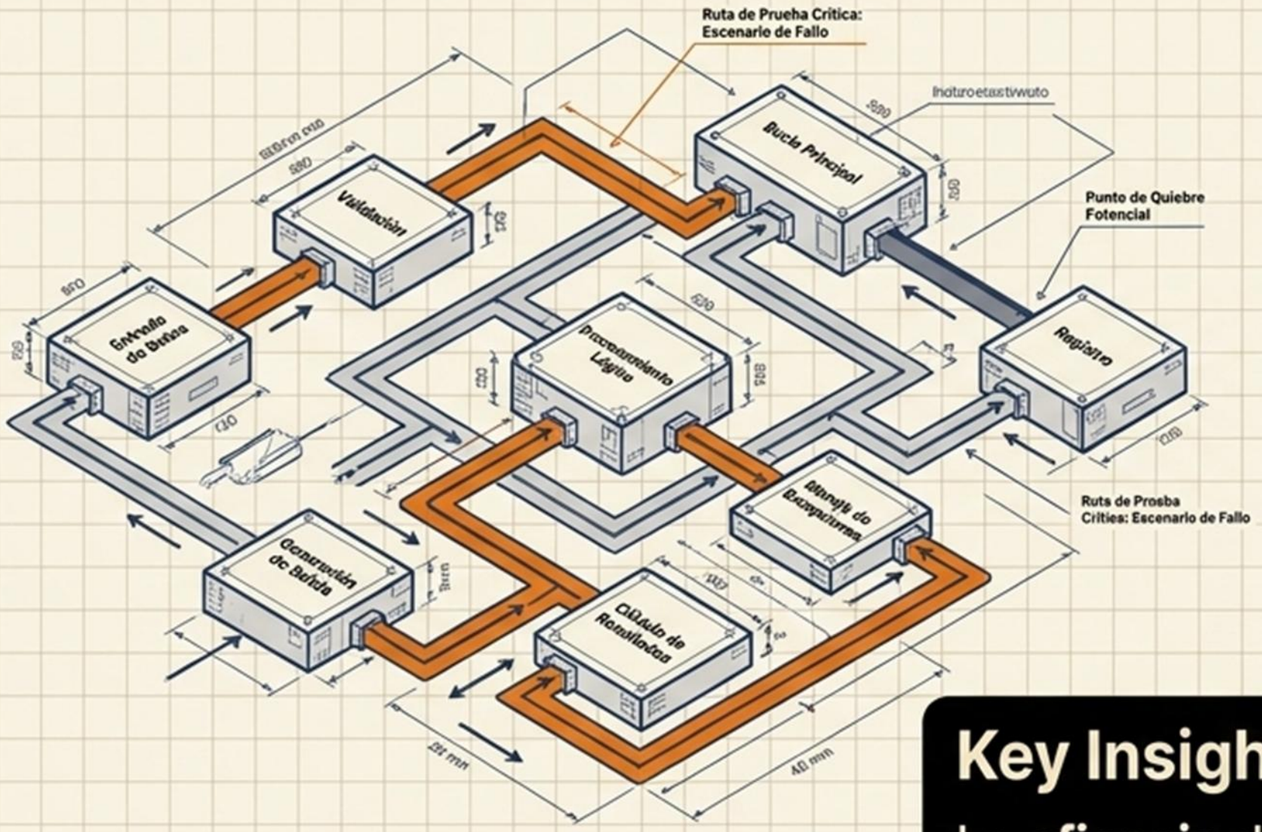


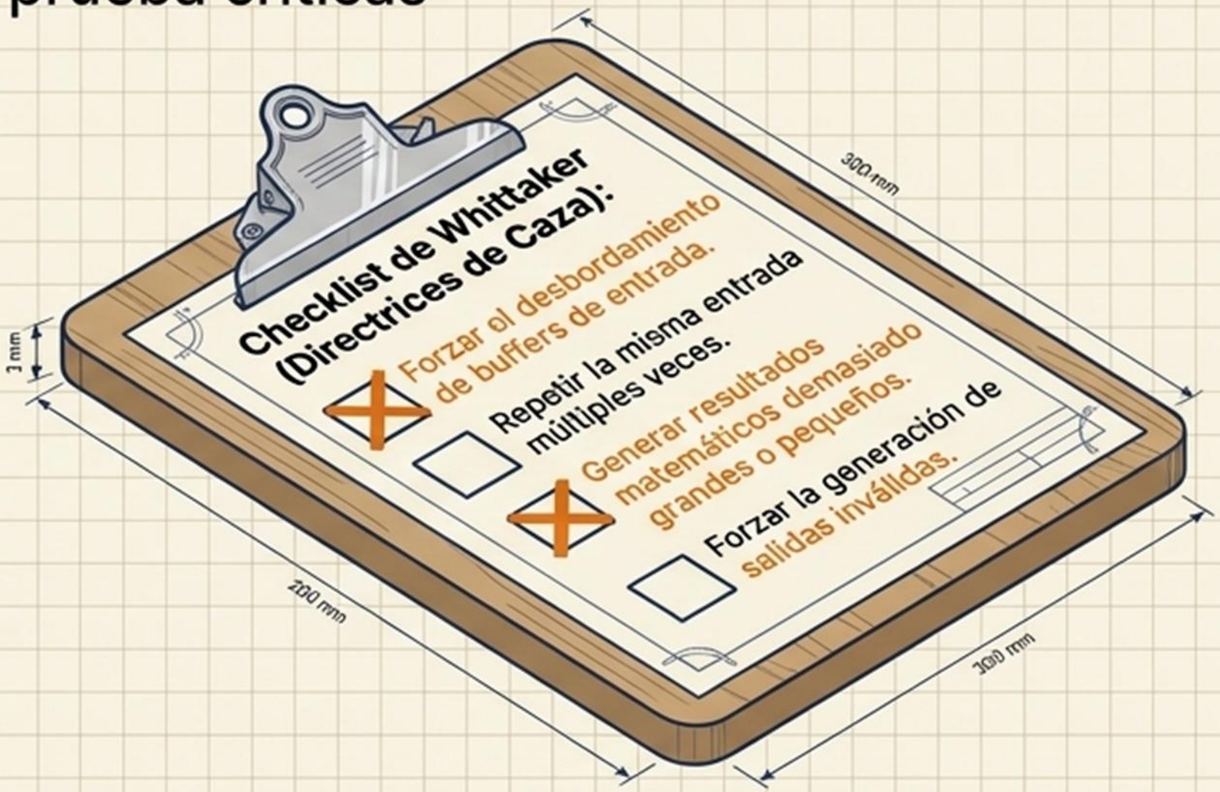
Regla de Inspección: No probar solo el punto medio. **Obligatoriamente probar:**

- El límite exacto inferior y superior.
- Un punto justo fuera del límite válido.
- Valores atípicos: Cero, colecciones vacías, y listas de un solo elemento.

Directrices y Pruebas de Trayectoria

Métodos para guiar la selección de rutas de prueba críticas y el diseño de casos de prueba efectivos.



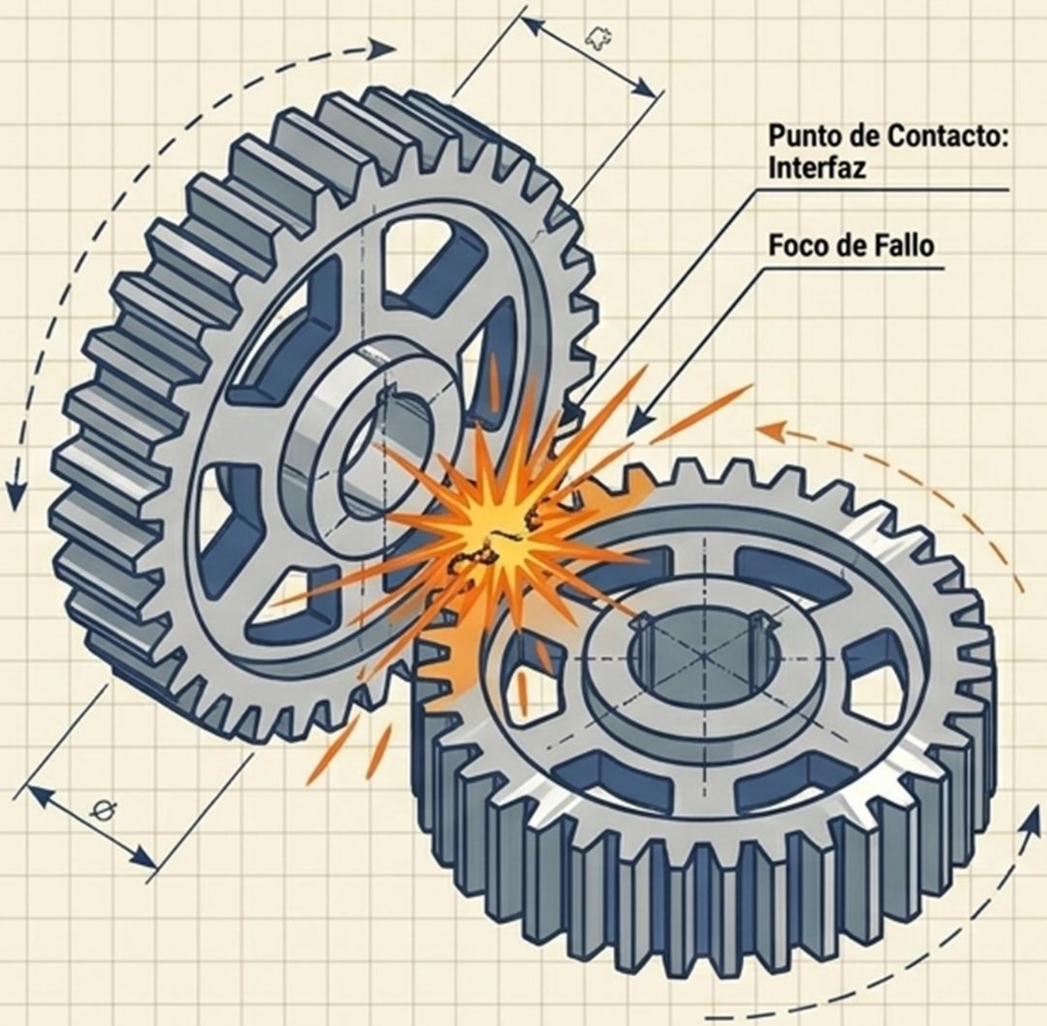


Key Insight

La **eficacia** de una **prueba** depende enteramente de la **astucia con la que se eligen los casos**. Ejecutar mil pruebas irrelevantes es menos útil que **una sola prueba en el punto crítico**.

Acto II: Pruebas de Componente e Integración

Estación 2. Evaluar grupos de unidades que han sido integradas.



El Foco Principal: Las Interfaces

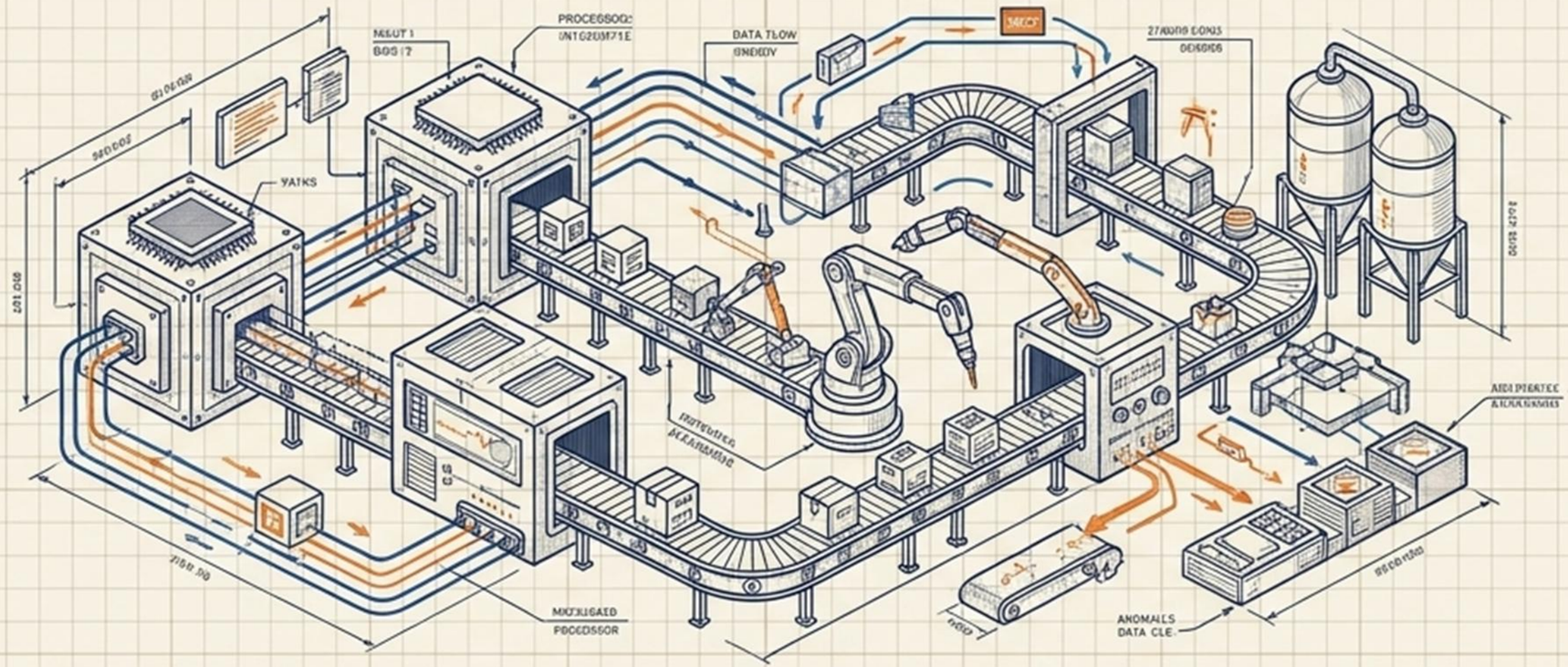
El lugar más probable donde habitan los defectos es en la conexión entre componentes perfectos por separado.

Tipos de Errores de Interfaz:

1. **Parámetros:** Datos incorrectos o en orden equivocado.
2. **Memoria Compartida:** Un componente sobrescribe los datos que otro componente necesita.
3. **Temporización:** Problemas de sincronización en sistemas de tiempo real.

La Estación Final Interna: Pruebas de Sistema

Se prueba el sistema completo ya integrado. El enfoque cambia de las interfaces individuales al comportamiento global.



Comportamiento Emergente

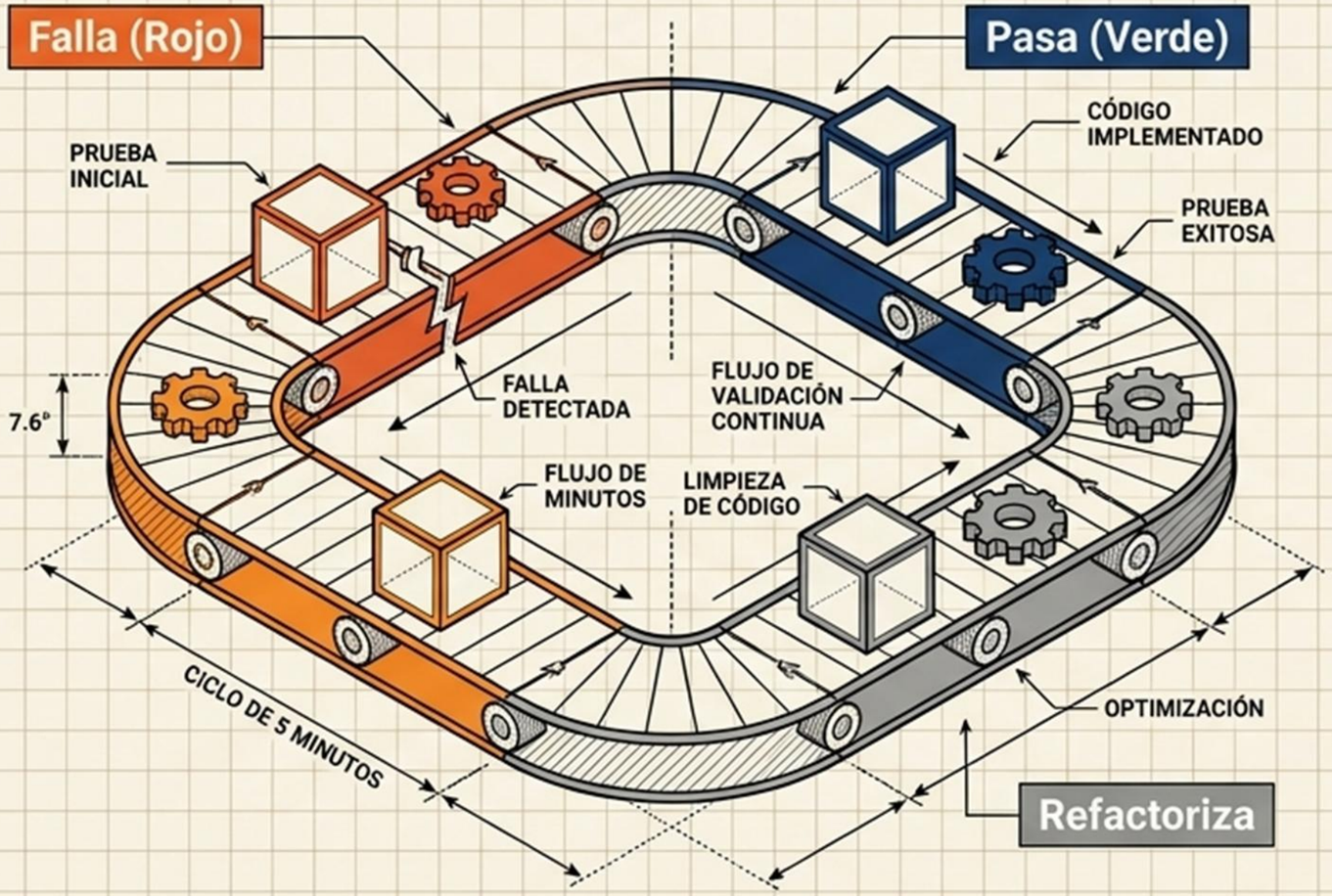
Existen características y defectos que solo se manifiestan cuando todas las piezas interactúan. Son **invisibles** probando las partes por separado.

Requerimientos No Funcionales

Es la fase para validar la confiabilidad, **seguridad y eficiencia estructural del sistema** como un todo **indivisible**.

Acto III: Desarrollo Dirigido por Pruebas (TDD)

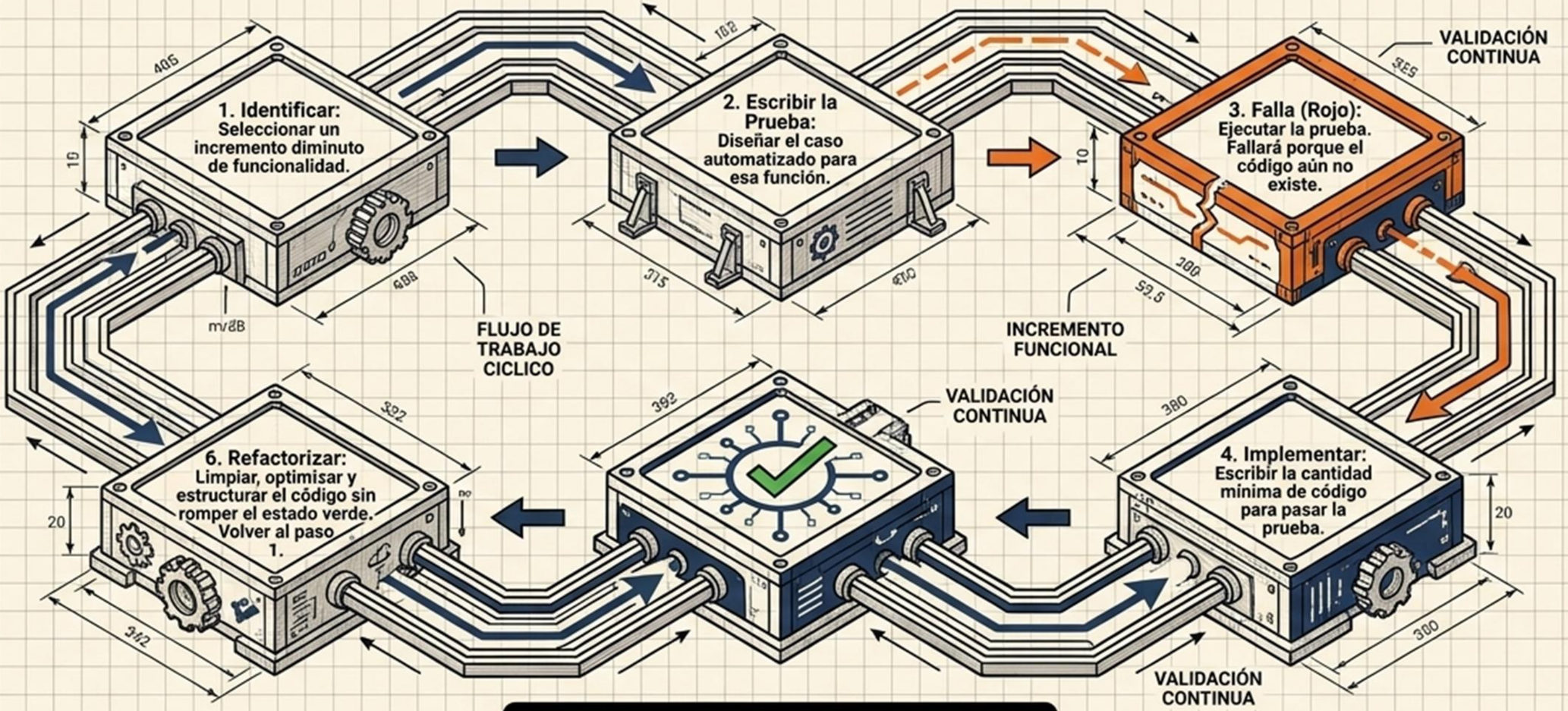
Invirtiendo el orden de la ingeniería: En lugar de escribir el código y luego probarlo, las pruebas automatizadas se escriben antes de que exista el código.



Beneficios Estructurales:

- **Cobertura Absoluta**: Cada línea de código está garantizada por una prueba.
- **Regresión Segura**: Si un cambio rompe algo, la cinta transportadora lo detecta en segundos.
- **Documentación Viva:** Las pruebas sirven como manual inquebrantable de lo que debe hacer el sistema.

La Maquinaria TDD en la Práctica



CONCLUSIÓN TDD:
El ciclo Red-Green-Refactor es el motor incesante que garantiza la calidad desde la primera línea de código.

Acto IV: Pruebas de Versión (Release)

Validación de versiones completas por un equipo de QA independiente antes de entregarlas al usuario. Es **estrictamente un proceso de Caja Negra**.

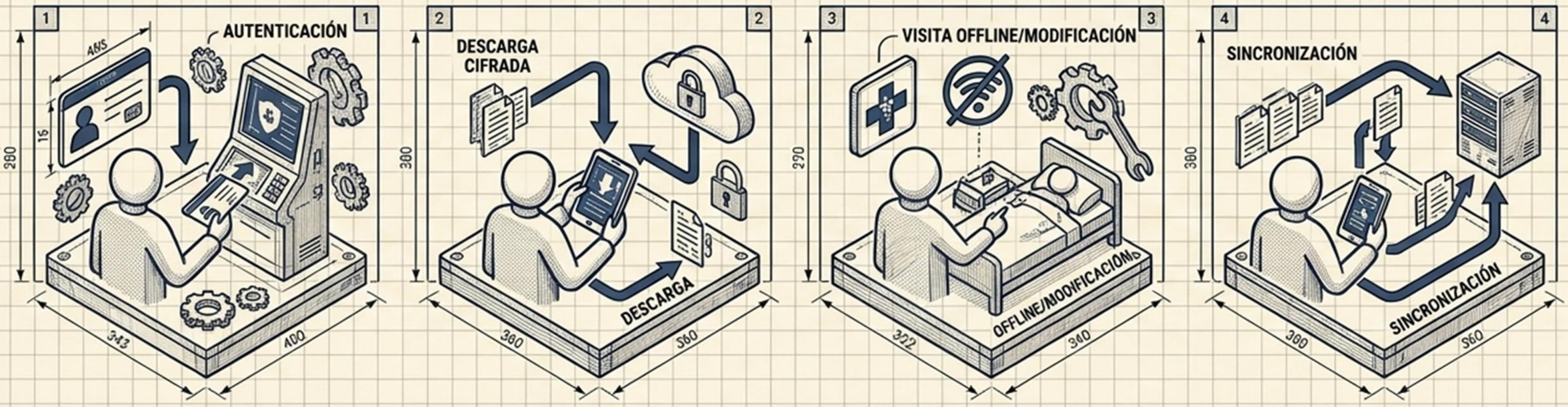
Pruebas Basadas en Requerimientos

- **Diseño** sistemático derivado directamente del documento maestro.

- **Regla de Oro:** Por cada requerimiento formal documentado (ej. "El sistema emitirá una alerta si el paciente es alérgico"), **debe existir al menos una prueba ejecutable** que demuestre su cumplimiento.

Pruebas de Escenario: El Mundo Real

Validar un requerimiento aislado no es suficiente. Los usuarios operan en secuencias y flujos narrativos.

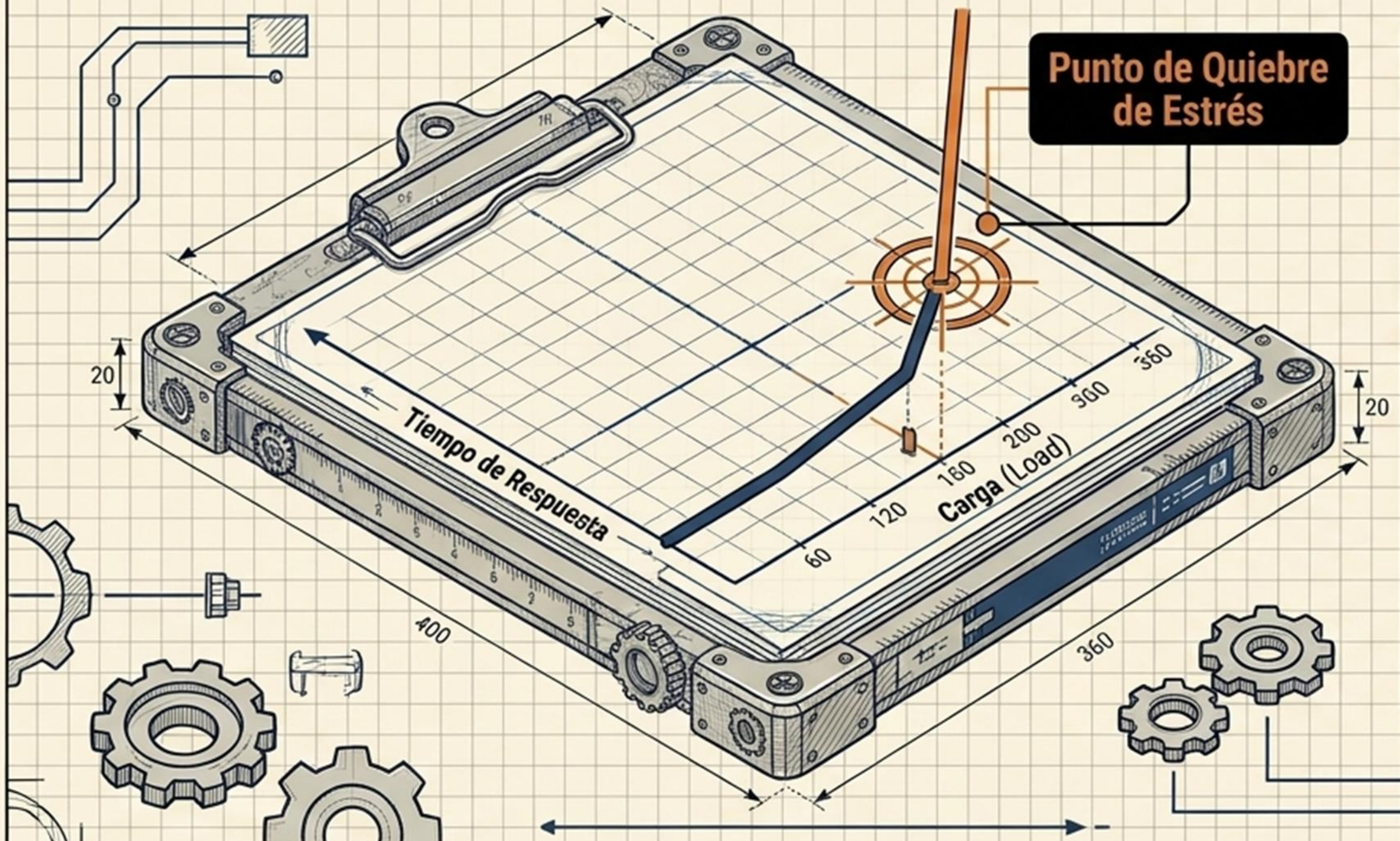


1. Autentica su usuario en la clínica.
2. Descarga registros cifrados al móvil.
3. Visita paciente offline y modifica dosis.
4. Sincroniza datos al volver a la red.

Por qué funciona: Obliga al sistema a comportarse en la secuencia desordenada y combinada del uso humano real, descubriendo **errores de estado o memoria** que pasan desapercibidos en las pruebas de unidades aisladas.

Pruebas de Rendimiento y Esfuerzo (Stress Testing)

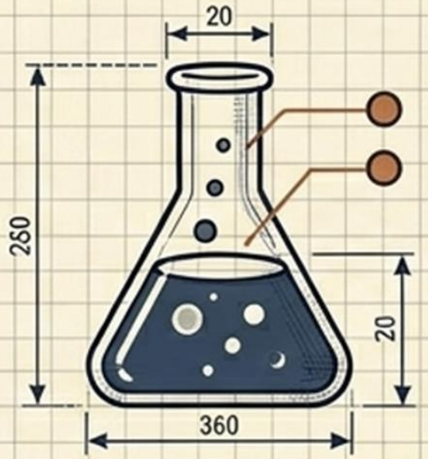
Someter el sistema a cargas que superan agresivamente sus especificaciones de diseño.



Objetivos del Estrés:

- No se trata solo de pasar el requerimiento, sino de **encontrar el punto exacto de colapso.**
- **Observar el modo de falla:** ¿El sistema se degrada elegantemente o los datos se corrompen de forma catastrófica?
- Identificar **cuellos de botella** emergentes bajo alta presión.

Acto V: Las Pruebas de Usuario

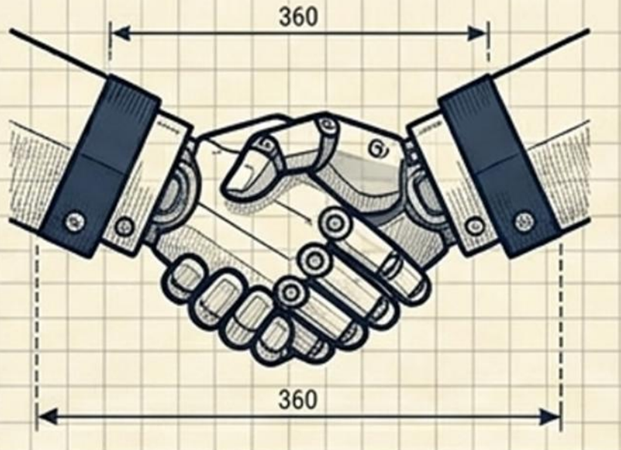


Pruebas Alfa

Usuarios potenciales operan el software dentro del entorno controlado del desarrollador. El equipo de ingeniería observa y registra anomalías en tiempo real.

Pruebas Beta

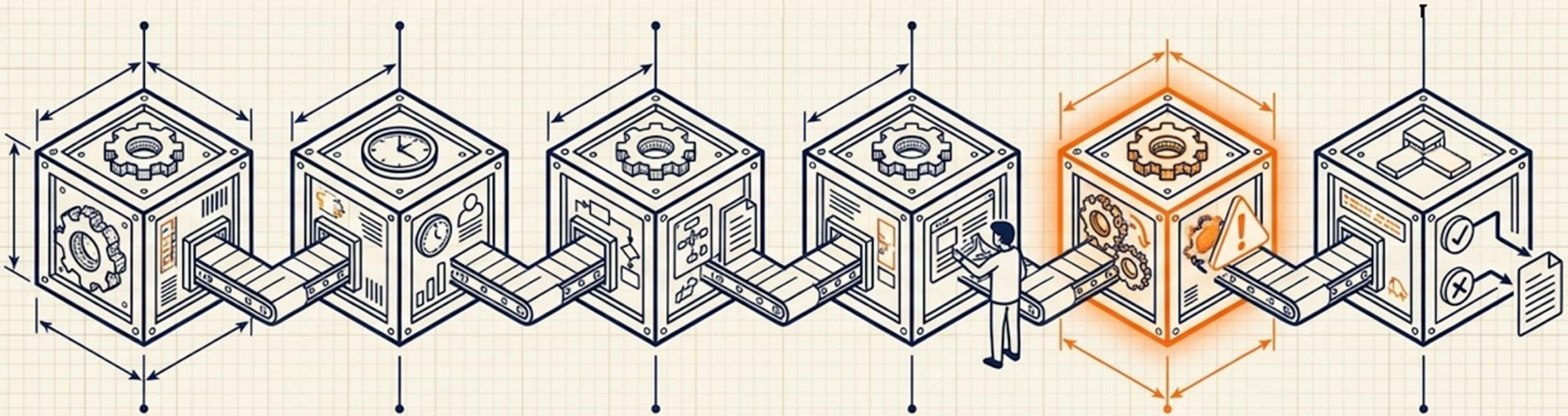
El software se libera a un grupo real en su propio entorno operativo. Sin supervisión directa; descubre fallas en hardware o flujos no anticipados.



Pruebas de Aceptación

Prueba formal y contractual donde el cliente verifica si el sistema cumple los criterios para ser liberado a producción masiva o facturado.

El Oleoducto de Aceptación



1. Definir Criterios:
Establecer qué significa "terminado".

2. Planificar:
Asignar tiempos y recursos del cliente.

3. Derivar Casos:
Crear pruebas desde los procesos de negocio.

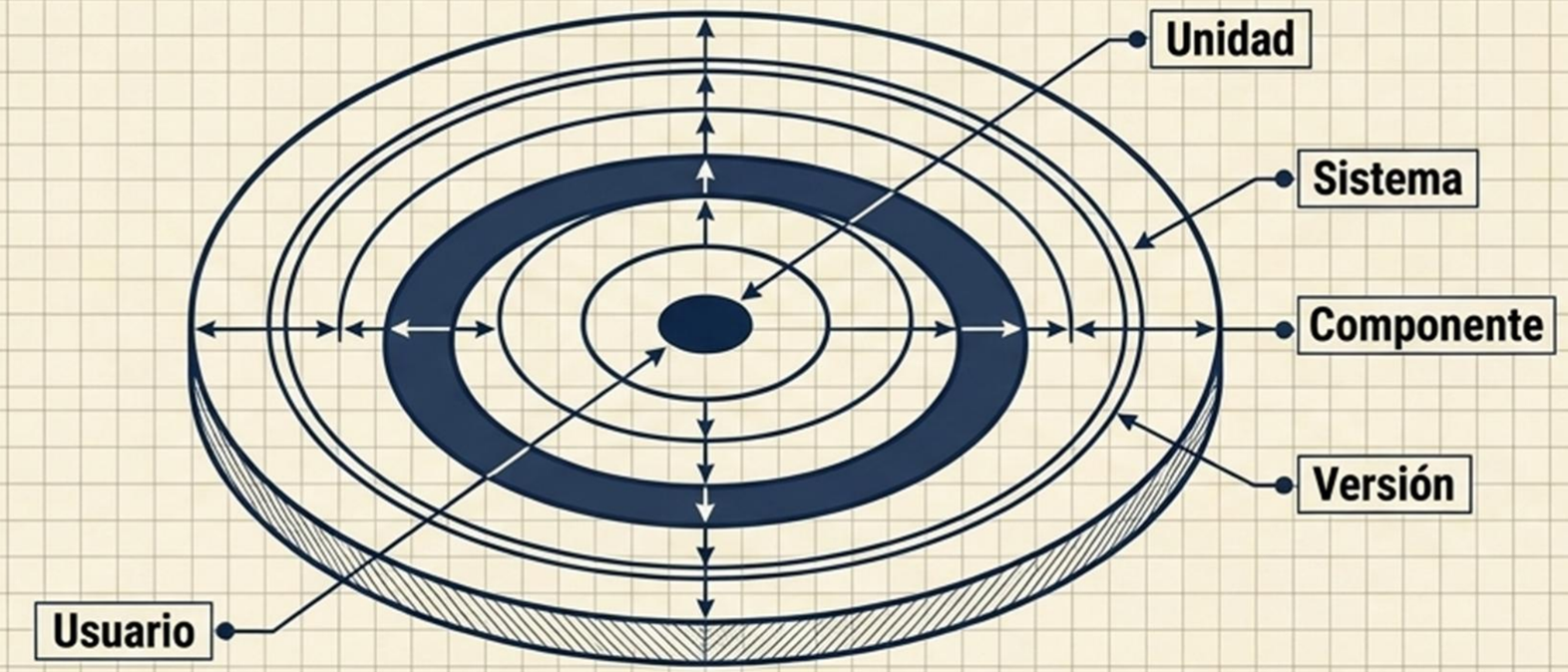
4. Ejecutar:
El cliente opera el sistema.

5. Negociar Resultados:
Revisar los defectos encontrados.
(El cliente puede descubrir que pidió algo que en la práctica no le sirve).

6. Aceptar/Rechazar:
Decisión final documentada.

El Mapa Completo

La cacería de defectos es una disciplina de expansión continua. Desde la **línea** de código microscópica hasta las **impredecibles manos del usuario**. El diseño arquitectónico de estas pruebas es lo que separa un programa funcional de una solución robusta.



Key Insight:
Las pruebas de usuario son esenciales aun cuando se hayan realizado pruebas abarcadoras previas, pues las pruebas en el entorno del desarrollador, forzosamente, son artificiales.

La Ingeniería del del Error

Estrategias y Técnicas de Diseño de Casos de Prueba

Tolerancia al fallo: 0.01%

SOFTWARE DEFECTS

NULL POINTER EXCEPTION

MEMORY LEAK

RESOURCE DRAIN

CONCURRENCY ISSUE

DATA
DATA
DATA
DATA
DATA
DATA

SECURITY VULNERABILITY

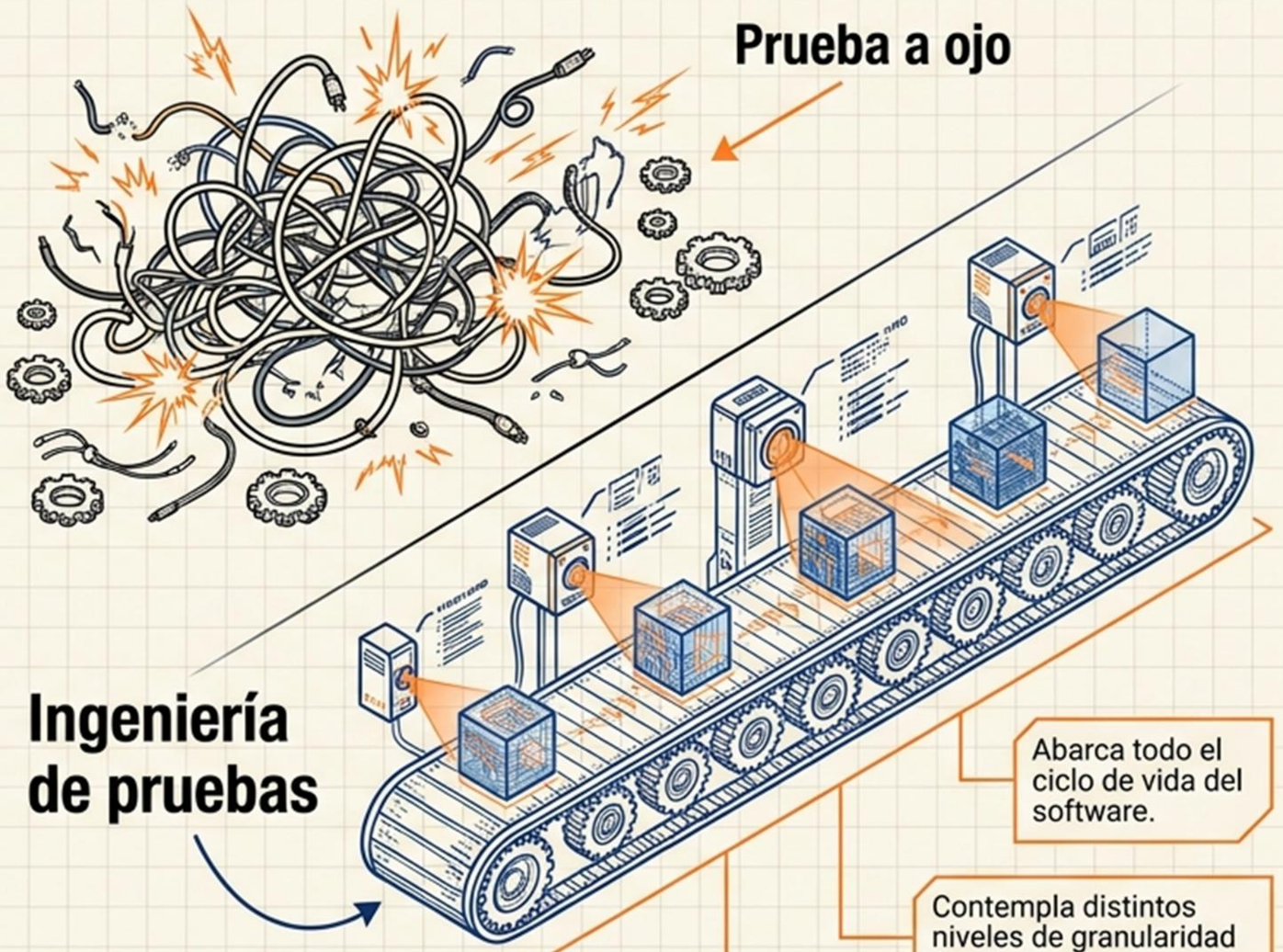
INJECTION POINT

Escala 1:1

UTN FACULTAD REGIONAL RAFAELA

El Enfoque Estratégico

Por qué probar no es simplemente “ejecutar el programa y ver qué pasa”.



KEY INSIGHT: Una estrategia de prueba sin un plan estructurado no es estrategia, **es esperanza.**

VERIFICACIÓN vs. VALIDACIÓN

VERIFICACIÓN

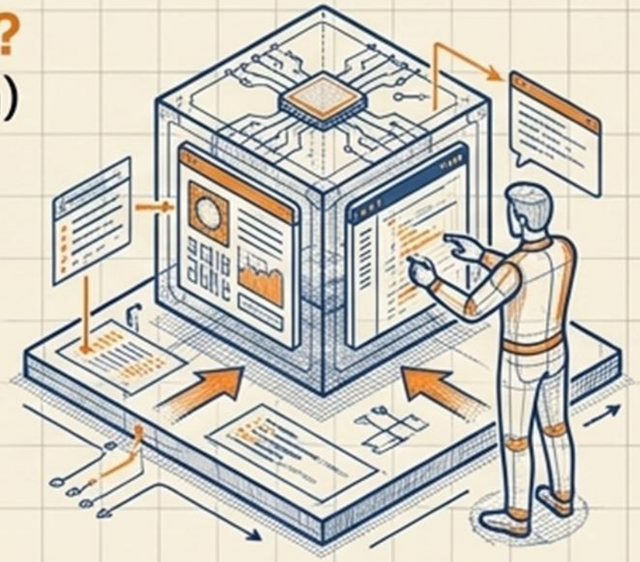
¿Construimos bien el producto?
(Pregunta de Boehm)

- Evaluación estática y dinámica.
- Revisiones técnicas, inspecciones de código.
- Revisa el código contra su diseño.

VALIDACIÓN

¿Construimos el producto correcto?
(Pregunta de Boehm)

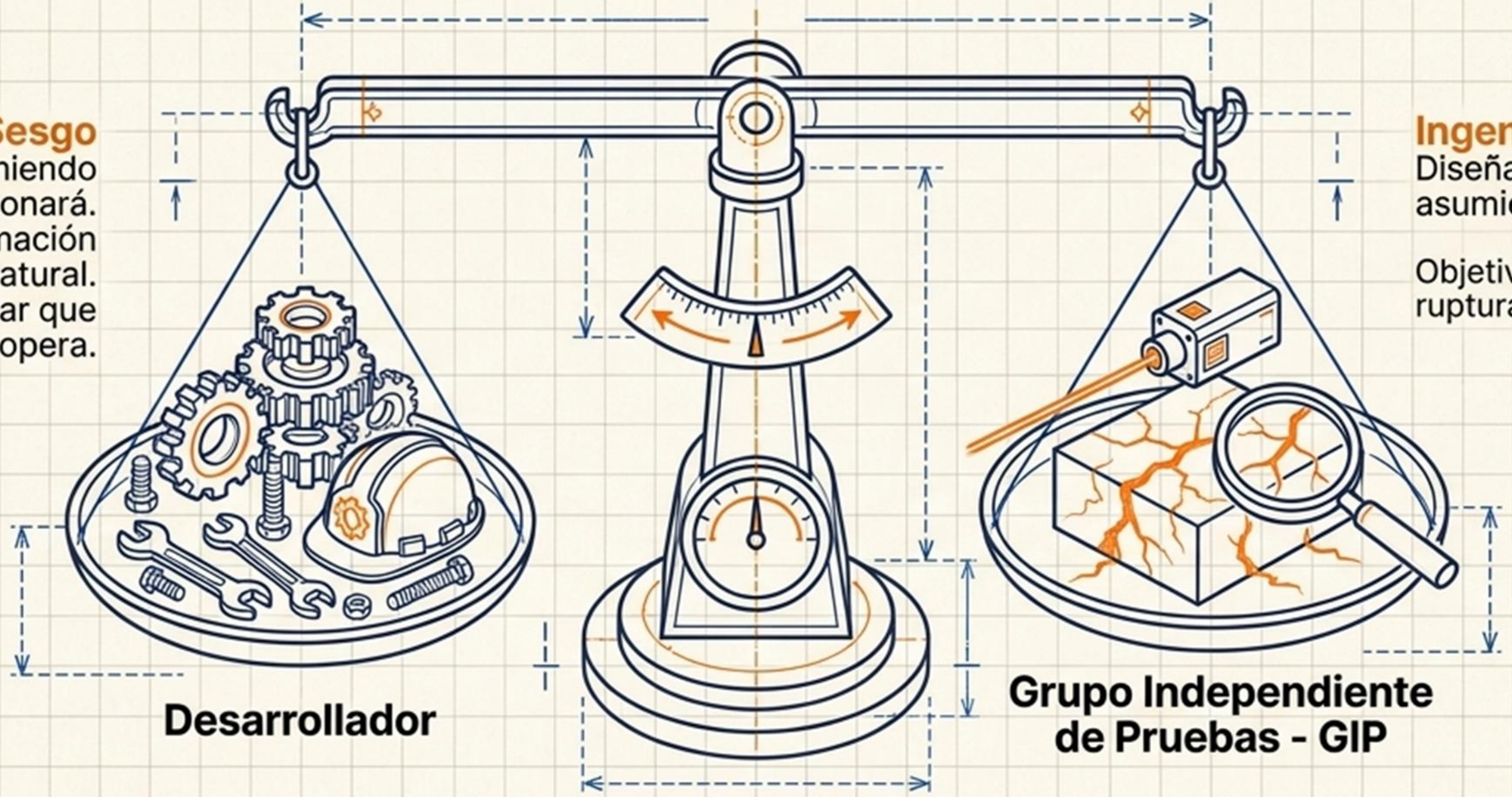
- Expectativas del cliente.
- El software cumple los requerimientos en el mundo real.



Auditorías de Configuración → Monitoreo de Rendimiento → Simulación → Testing Estricto

El Conflicto Psicológico

Tensión de Sesgo
Construye asumiendo que funcionará.
Sesgo de confirmación natural.
Objetivo: Demostrar que la unidad opera.



Desarrollador

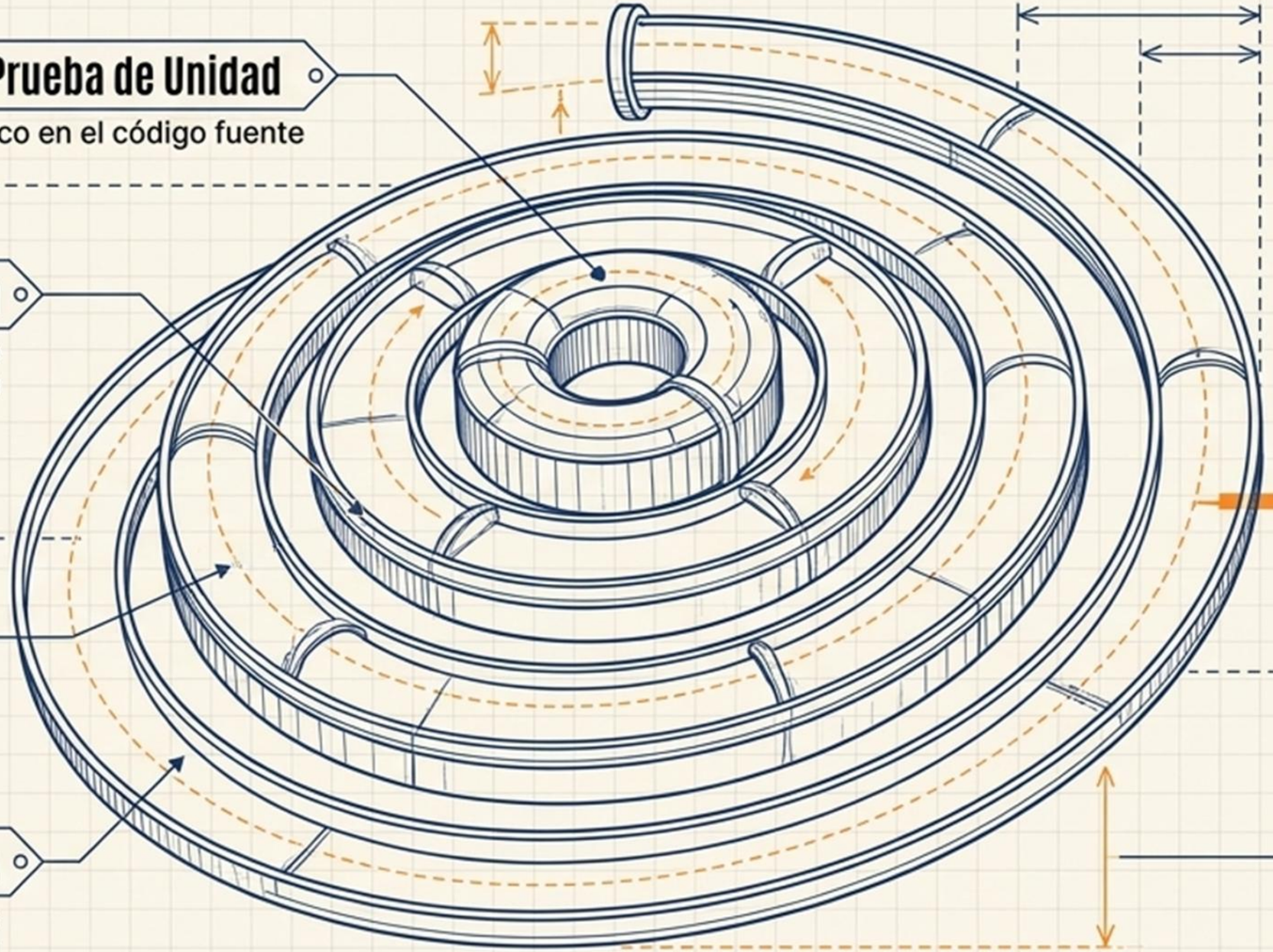
Grupo Independiente de Pruebas - GIP

Ingeniería de Fractura
Diseña pruebas asumiendo fallos.

Objetivo: Encontrar la ruptura sistemáticamente.

KEY INSIGHT: El conflicto de intereses es inevitable: quien construye un sistema rara vez posee la crueldad metódica necesaria para destruirlo.

Acto I — La Estrategia en Espiral



Prueba de Unidad
Foco en el código fuente

Prueba de Integración
Foco en el diseño de arquitectura

Prueba de Validación
Foco en el análisis de requerimientos

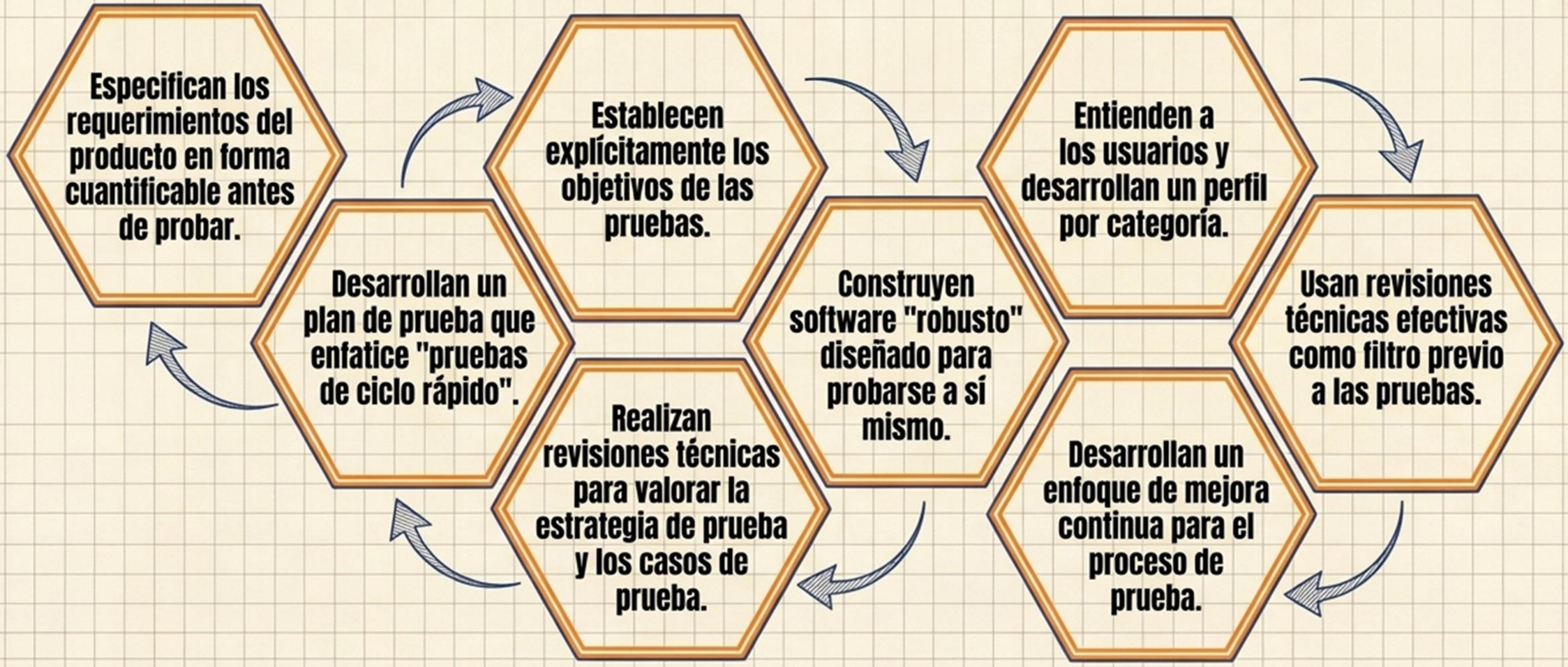
Prueba del Sistema
Foco en la ingeniería de sistemas global

Criterios STOP:
Una prueba termina no cuando se acaban los errores, sino cuando se cumplen los criterios predefinidos de cobertura o métricas de fiabilidad.

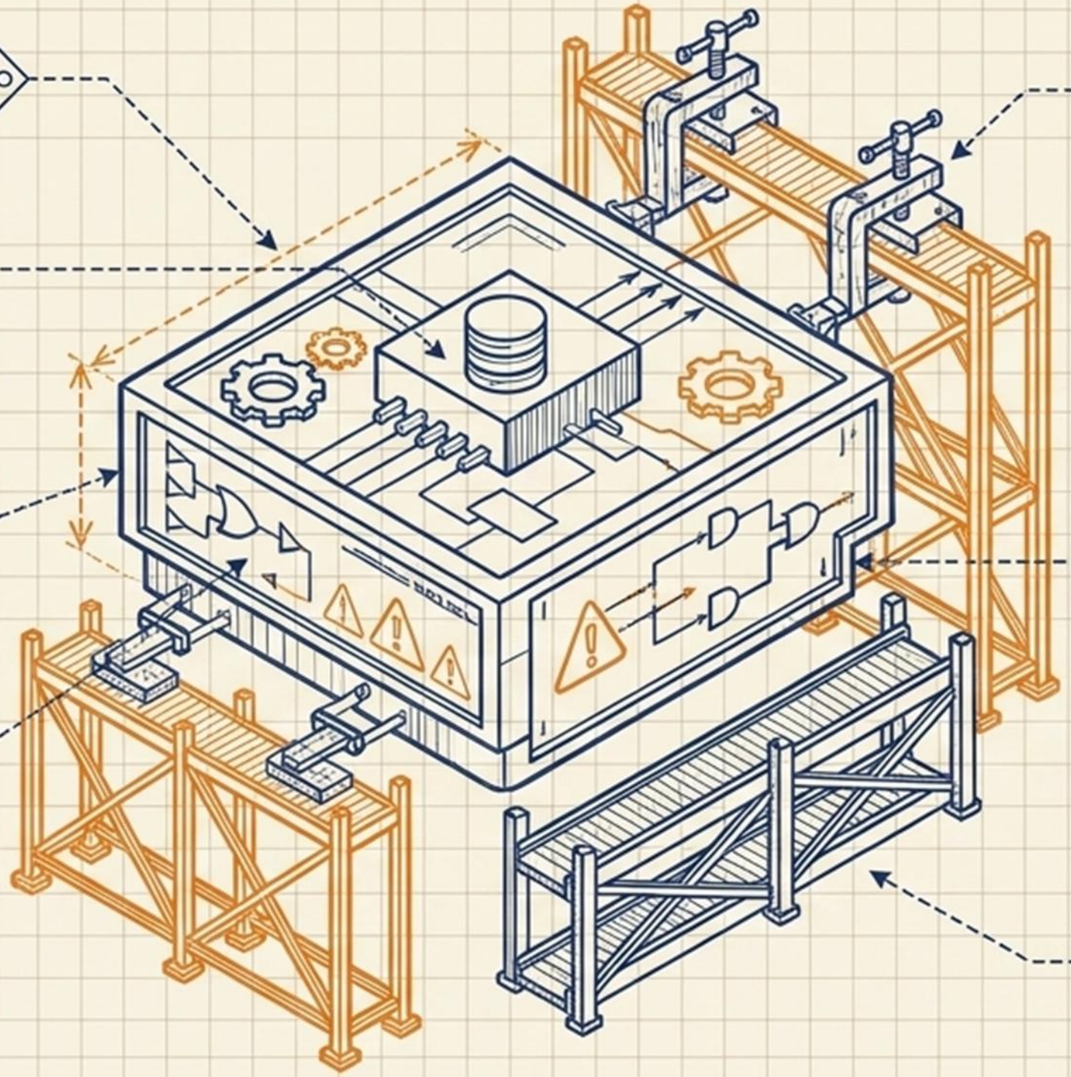
UTN FACULTAD REGIONAL RAFAELA

Aspectos estratégicos críticos

los ocho puntos que toda estrategia de prueba debe responder:



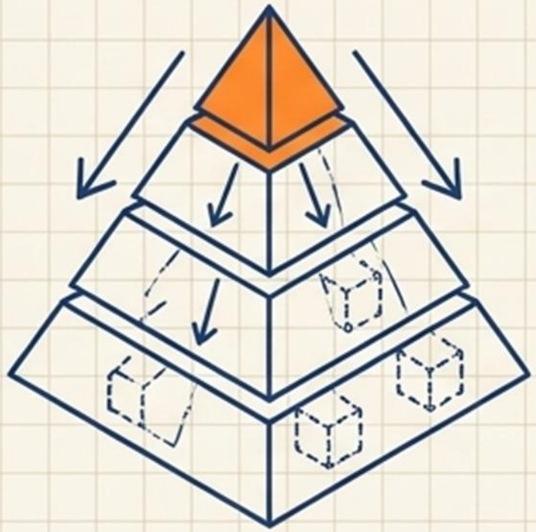
El Nivel Atómico: Prueba de Unidad



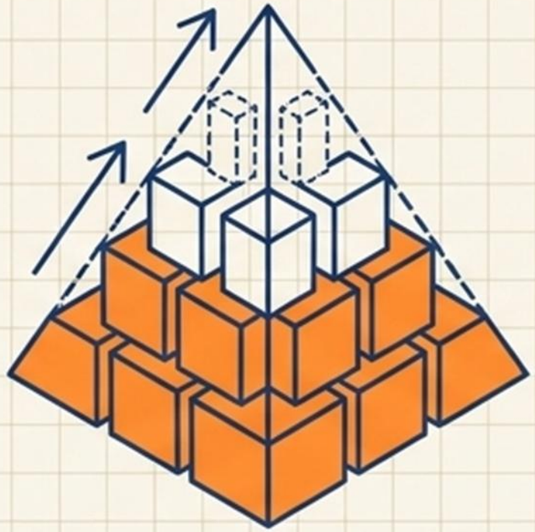
- Interfaces del módulo.
- Estructuras de datos locales.
- Caminos lógicos independientes.
- Caminos de manejo de errores.
- Drivers (Módulos principales simulados)
- Condiciones de frontera.
- Stubs (Módulos subordinados simulados)

Pruebas de Integración: Evitando el Caos del "Big Bang"

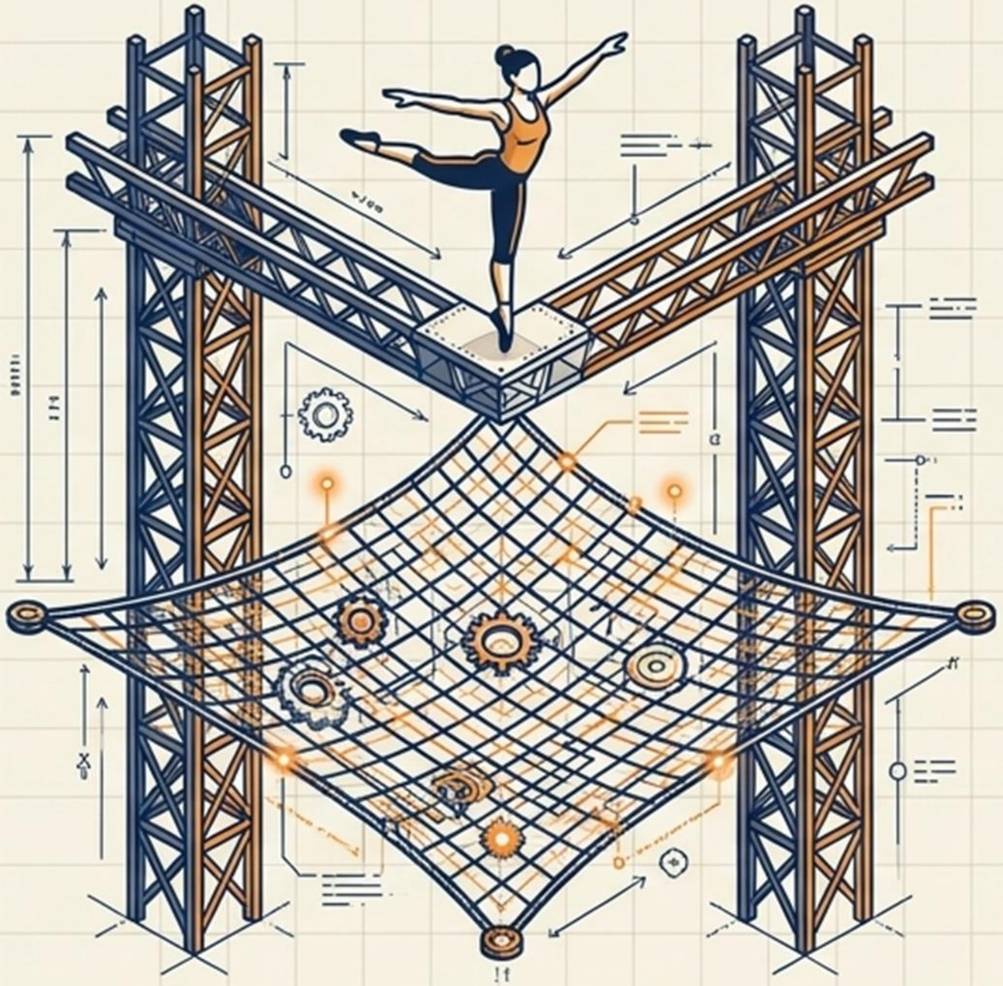
Juntar todo y probar de golpe garantiza el fallo y oculta la causa raíz.
La solución es la Integración Incremental.



Integración Descendente (Top-Down)	Integración Ascendente (Bottom-Up)
Mecánica: Empieza en el control principal. Sustituye stubs progresivamente.	**Mecánica:** Empieza por módulos atómicos. Usa drivers para agrupar (“clusters”).
Pro: Verifica decisiones lógicas mayores rápido.	**Pro:** No necesita stubs; procesamiento básico se prueba temprano.
Contra: Necesita muchos stubs; aplaza pruebas de I/O.	**Contra:** El programa no existe como entidad hasta el final.



La Red de Seguridad: Pruebas de Regresión



El Riesgo de la Entropía

Cada cambio, adición o corrección es un riesgo inminente de ruptura (efectos colaterales).

El Concepto de Regresión

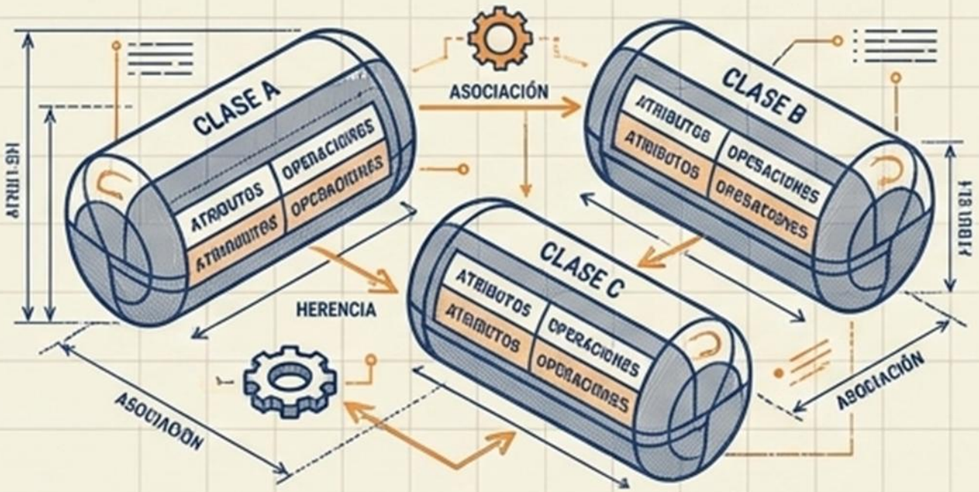
Un subconjunto crítico de pruebas que se re-ejecuta ante cada modificación para asegurar que lo que funcionaba sigue funcionando.

Conexión Operativa

Es la base de las “Pruebas de Humo” (Smoke testing) y la Integración Continua. Si la prueba de humo falla, el latido del proyecto se detiene.

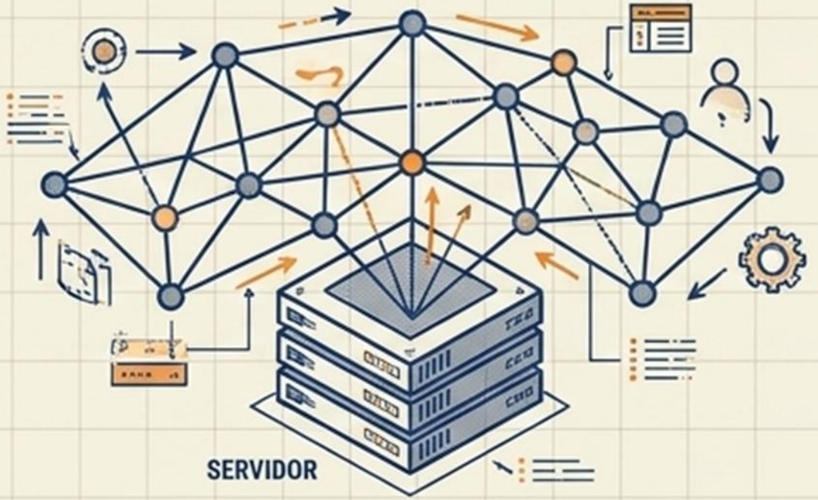
Cambio de Paradigma: Orientación a Objetos y Web

Orientación a Objetos



- **Cambio de Unidad:** La unidad ya no es el método, es la Clase completa (atributos + operaciones).
- **Focos Clave:** Prueba de operaciones de la clase, transiciones de estado, y el efecto de la herencia (la operación heredada debe probarse en el nuevo contexto).

WebApps



- **Naturaleza Dinámica:** Se prueba la experiencia fluida y la arquitectura de red.
- **Dimensiones de Prueba:** Contenido, Interfaz de Usuario, Navegación, Rendimiento (stress), Seguridad y Configuración de entorno.

El Choque con la Realidad: Validación y Sistema

Pruebas de Validación

Objetivo:
Verificar que el software cumple los requerimientos del cliente (SRS).

Resultados posibles:
Cumplimiento o Desviación registrada.
(Incluye Alfa y Beta Testing).

Pruebas del Sistema (Entorno Exterior)

Recuperación:
Forzar fallos para medir tiempo de restauración.

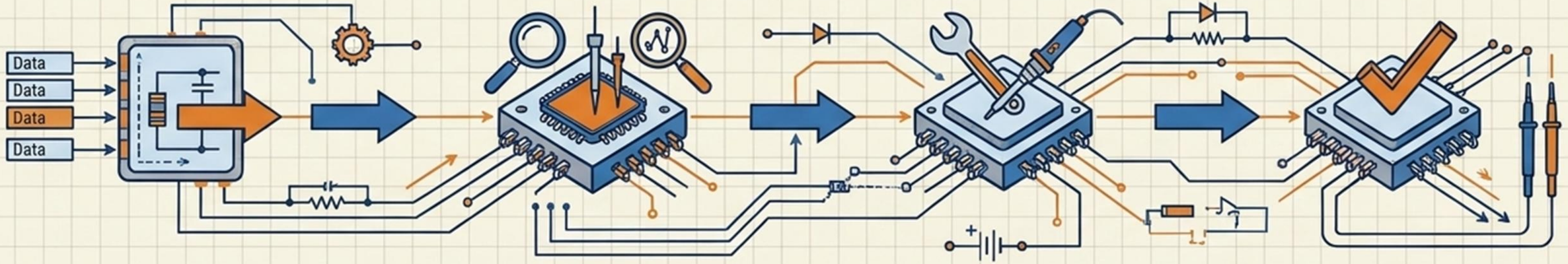
Seguridad:
Defensas contra penetración interna/externa.

Esfuerzo (Stress):
Demanda anormal de recursos hasta la ruptura.

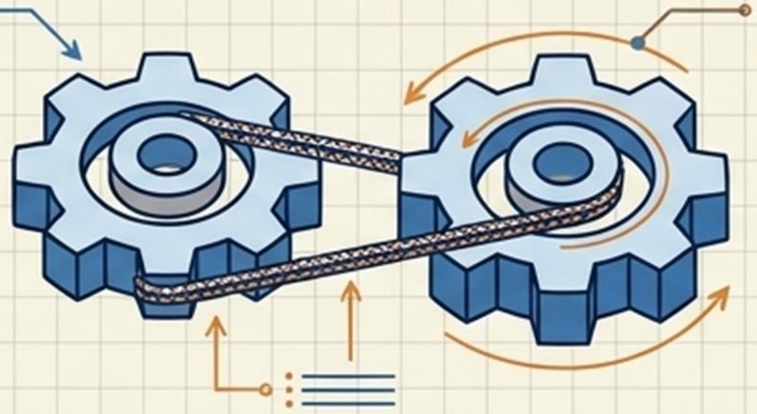
Rendimiento:
Tiempos de ejecución en contexto del sistema global.

Diagnóstico Clínico: El Arte de Depurar

Probar encuentra el síntoma. Depurar aísla la causa.



Fuerza Bruta
Volcado de memoria, prints masivos. Menos eficiente.



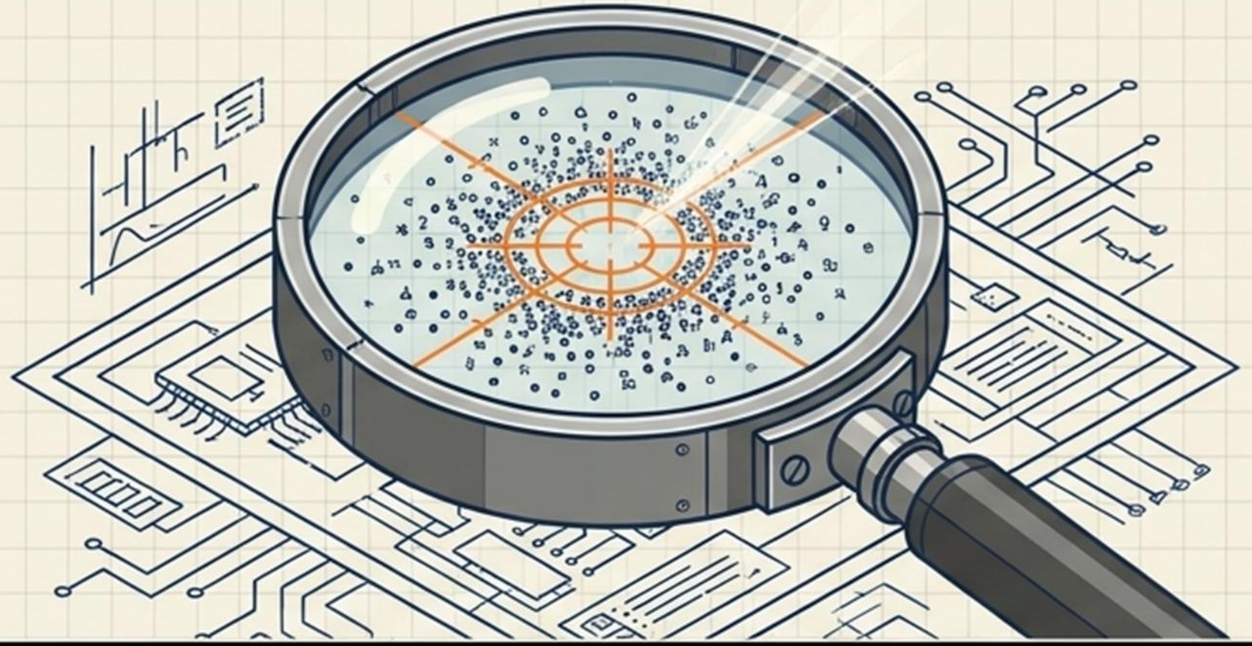
Rastreo hacia Atrás
Seguir hilo lógico desde síntoma al inicio. Útil en programas pequeños.

Eliminación de Causas
Método científico: hipótesis, prueba, descarte.

Acto II — Fundamentos del Diseño de Casos de Prueba

Prueba Exhaustiva

Ejecutar todas las permutaciones posibles es matemáticamente imposible.



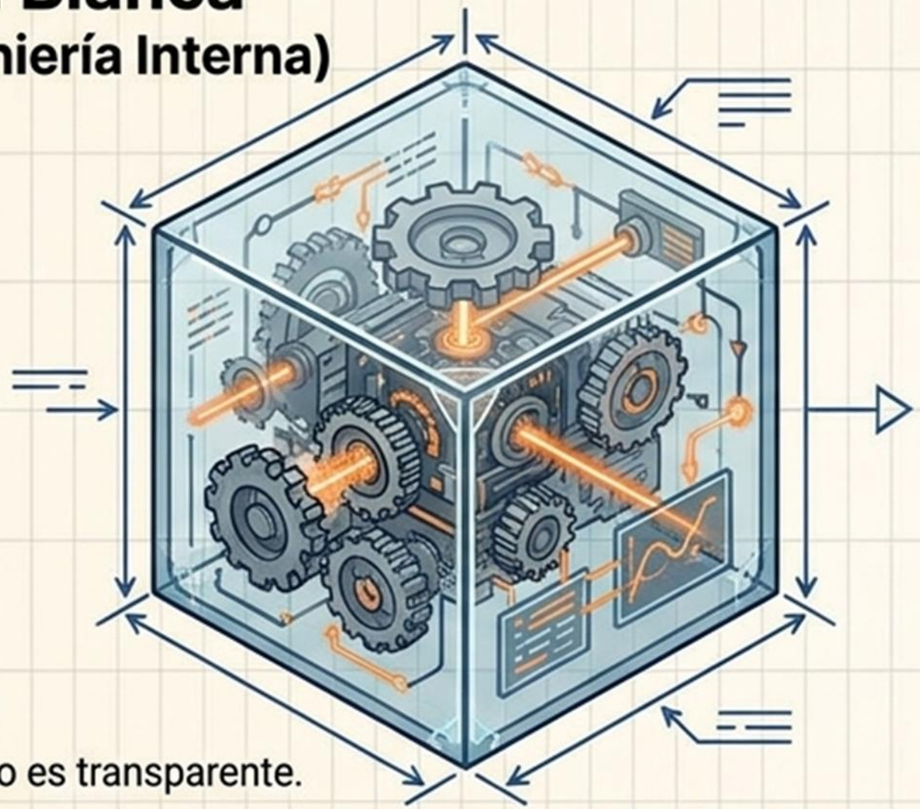
Prueba Representativa

Seleccionar un subconjunto **ínfimo** que logre la máxima cobertura estadística. El verdadero arte de la ingeniería de pruebas.

KEY INSIGHT: El objetivo de un caso de prueba excelentemente diseñado es encontrar el mayor número de errores con la menor inversión de tiempo y esfuerzo.

Las Dos Filosofías: Caja Blanca vs. Caja Negra

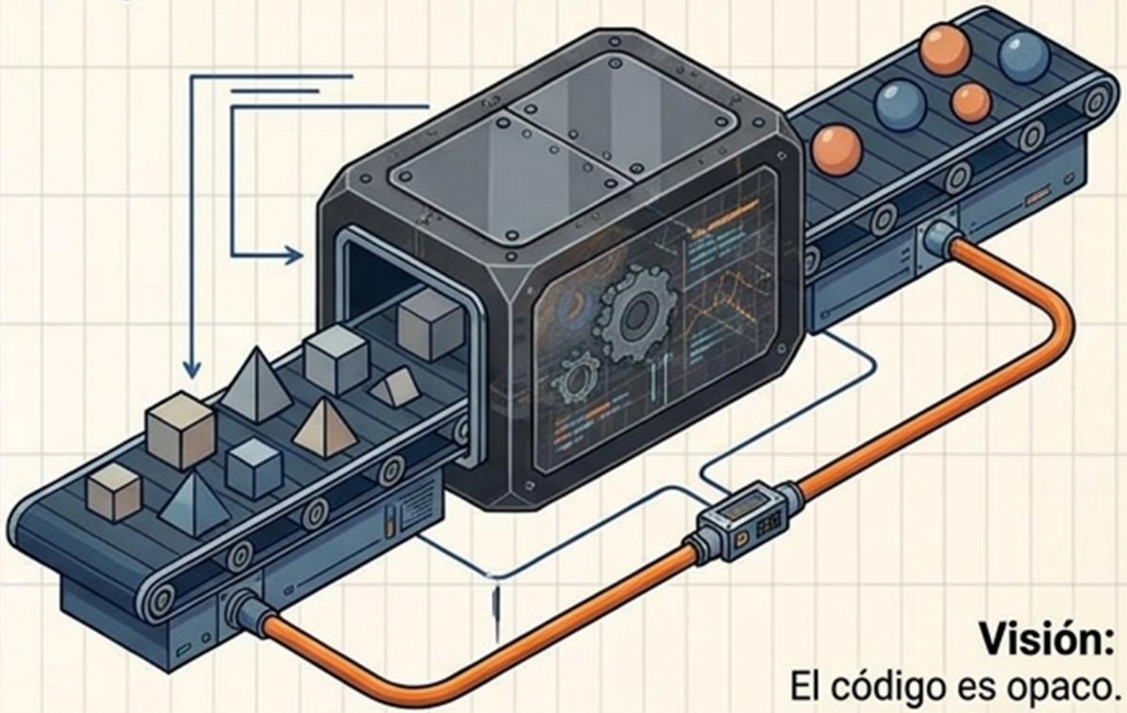
Caja Blanca
(Ingeniería Interna)



Visión:
El código es transparente.

Foco:
Estructura de control. Garantiza que todos los caminos independientes y sentencias lógicas se ejecutan al menos una vez.

Caja Negra
(Ingeniería Externa)



Visión:
El código es opaco.

Foco:
La función y los requerimientos. Valida entradas contra salidas esperadas sin conocer la implementación interna.

KEY INSIGHT:
Ninguna reemplaza a la otra. Son engranajes complementarios que se cruzan en el aseguramiento de la calidad.

UTN FACULTAD REGIONAL RAFAELA

Acto III — Técnicas de Caja Blanca: Prueba de Ruta Básica

Complejidad Ciclomática: V(G) = Aristas - Nodos + 2

Define el límite superior del número de caminos lógicos independientes.

Protocolo

1. Dibujar el grafo de flujo desde el código fuente.
2. Calcular métrica V(G).
3. Identificar cada ruta base linealmente independiente.
4. Diseñar un caso de prueba obligatorio por cada ruta.

Expandiendo la Caja Blanca: Pruebas de Estructura de Control

1. Prueba de Condición

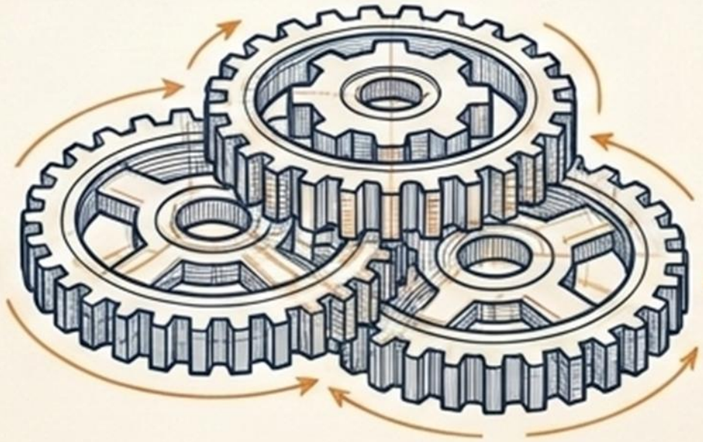
Escrutinio lógico. Ejercitar exhaustivamente cada condición lógica individual (verdadero/falso) dentro del módulo para atrapar errores booleanos.

2. Prueba de Flujo de Datos

Rastreo molecular. Seguir la trayectoria de una variable desde su punto de definición explícita hasta cada punto donde se utiliza.

3. Prueba de Bucle

Foco en la ejecución de bucles. Verificar la corrección de las estructuras de repetición probando diferentes números de iteraciones, incluyendo casos límite y típicos.

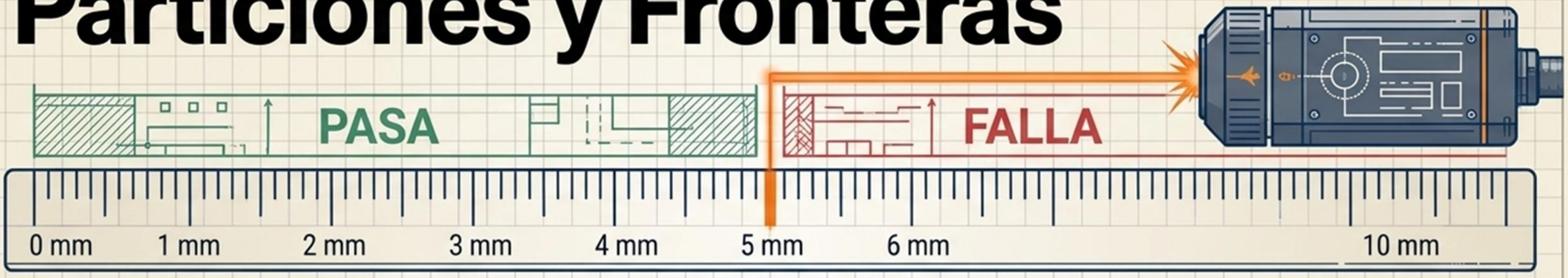


Protocolo para bucles simples, anidados y concatenados.

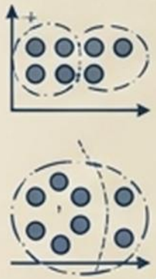
Valores críticos obligatorios

Saltarse el bucle, 1 pasada, 2 pasadas, m pasadas (m < n), n-1 pasadas, n pasadas.

Acto IV — Técnicas de Caja Negra: Particiones y Fronteras



Partición de Equivalencia



Agrupar el vasto dominio de entrada en clases equivalentes válidas e inválidas. Probar un representante de cada clase es estadísticamente suficiente.

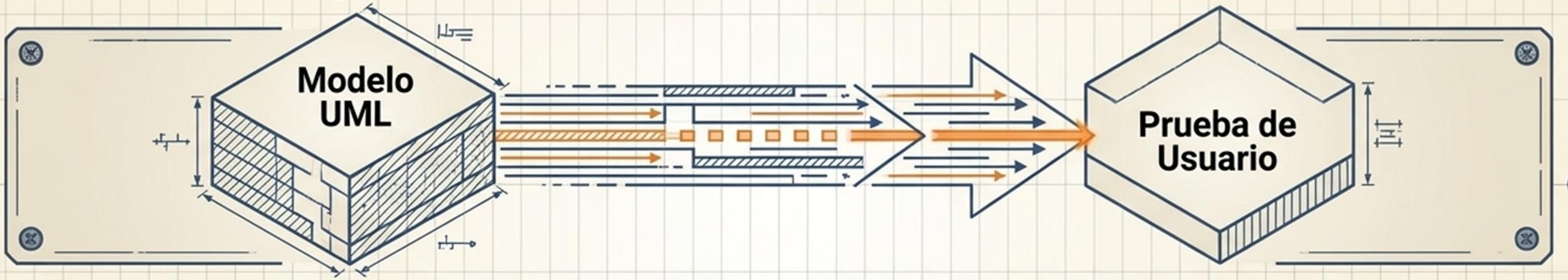
Análisis de Valores de Frontera (BVA)

Las reglas cambian aquí. Forzar pruebas exactamente en el límite de la partición, justo por debajo y justo por encima.

KEY INSIGHT: La mayoría de los defectos se ocultan en los bordes de la lógica, no en el centro. Los errores viven en las fronteras.

UTN FACULTAD REGIONAL RAFAELA

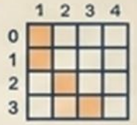
Pruebas Basadas en Modelos (PBM) y Métodos Especializados



Prueba Basada en Modelo

Usar diagramas UML (casos de uso, diagramas de estado) para derivar de forma automatizada y sistemática las secuencias lógicas de las pruebas.

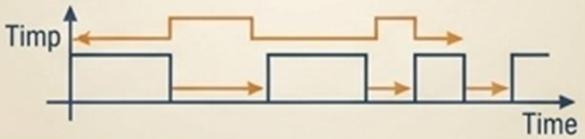
Prueba de Arreglo Ortogonal



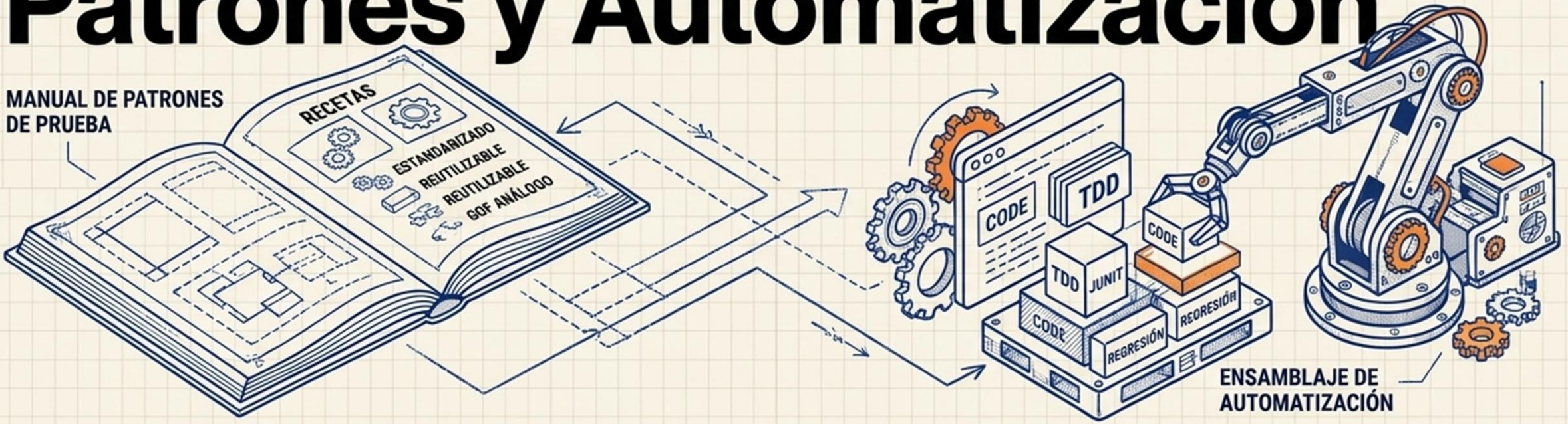
Herramienta matemática. Reduce el universo de miles de combinaciones (ej. GUI complejos) a una matriz pequeña altamente efectiva que encuentra errores interaccionales.

Sistemas Especiales

En sistemas de tiempo real, el foco absoluto recae sobre la temporalidad (eventos/tiempo) más allá de la mera función.



Escalabilidad Tecnológica: Patrones y Automatización

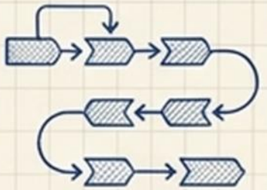


Patrones de Prueba

Soluciones de diseño estandarizadas y reutilizables para problemas de prueba recurrentes en arquitecturas complejas (análogo a patrones GoF).

La Necesidad de Automatización

Fundamental para TDD (Test-Driven Development). Marcos de trabajo como JUnit y la ejecución continua de scripts de regresión.



El Límite de la Máquina

La automatización no descubre cosas nuevas, sólo evita regresiones. El diseño del caso de prueba siempre requiere intelecto humano.

La Cacería Incesante

KEY INSIGHT FINAL: Las **pruebas nunca terminan:** sólo se transfieren del ingeniero al cliente. **Diseñar con precisión desde el inicio** es la única forma de que esa transferencia no termine en un desastre.